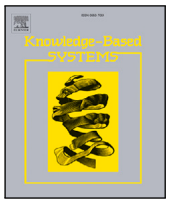
Knowledge-Based Systems 352 (2026) 117001



Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys





Vulnerabilities in autonomous execution: A survey of security threats and defenses in LLM-driven agentic and multi-agent systems

Vincenzo Sammartino

Dipartimento di Informatica, Università di Pisa, Largo B. Pontecorvo 3, 56127 Pisa, Italy

ARTICLE INFO

Keywords:
LLM agents
Multi-agent systems
Prompt injection
AI security
Information-flow control
systematic literature review

ABSTRACT

Large language models (LLMs) are no longer confined to producing text. Equipped with tools and composed into multi-agent systems, they now *act* on the world with limited human oversight. This shift from generation to autonomous execution transforms the security landscape. While prompt injection against a chatbot yields, at worst, an embarrassing sentence, the same injection against a tool-using agent can result in an exfiltrated database, a fraudulent transaction, or a poisoned message propagating to peer agents. The vulnerability class is not new, but the blast radius is.

This article surveys the security of LLM-driven agentic and multi-agent systems from 2023 through early 2026, following a pre-specified, PRISMA-informed review protocol that we report in full so that the corpus can be reproduced and extended. We contribute a threat taxonomy classifying attacks by *entry point*, *propagation path*, and *target asset*, together with an explicit statement of the conditional dependencies among these three dimensions. Using this, we organise four threat families – indirect prompt injection, malicious tool use, inter-agent infection, and memory poisoning – consolidating 44 attack papers into a unified comparison, annotated with reported attack success rate, paired task utility, and peer-review status. We review the defensive landscape across five categories (detection, structural separation, execution isolation, information-flow control, and alignment-side hardening) and map defenses to threats under an operationally defined coverage criterion, assessing each family for adaptive-attack resistance, deployment cost, and residual risk, to expose coverage gaps. Two findings recur. First, no purely prompt-level defense in the reviewed literature is reported to withstand adaptive attackers; effective security requires constraining what the *system* around the model is permitted to do, rather than relying on model robustness. Second, multi-agent propagation – the mechanism making agentic compromise qualitatively worse than single-model compromise – is the least defended part of the stack in the corpus we assembled, with almost no deployed frameworks enforcing trust boundaries between cooperating agents. We conclude with a research agenda and a minimal security-reporting standard for agent frameworks.

1. Introduction

For most of the short history of large language models, their security problems were bounded by their output channel. A model that could be persuaded to write a phishing email had produced a string; a human still had to send it. A model that leaked a system prompt had leaked a paragraph. The attacks were real and the disclosures were embarrassing, but the damage was mediated—at every step there was a person deciding whether to act on the text.

That mediation is being removed. Since 2023, the dominant architectural pattern in applied LLM research has been the *agent*: a model placed in a loop where it observes an environment, reasons about a goal, and invokes tools that change external state [1–4]. These tools are not confined to sandboxes. Production agents read and send email, query and write to production databases, execute shell commands, open pull requests, browse arbitrary websites, and move money. The Model Context Protocol [5] and comparable interfaces have made connecting an LLM to an arbitrary third-party service a configuration exercise rather than an engineering project. And increasingly these agents are not solitary: frameworks such as AutoGen [6], MetaGPT [7], and ChatDev [8] compose many agents into systems where the output of one becomes the input – the *trusted* input – of the next.

Each of these developments is a capability win and a security loss, and the losses compound in a way the security literature has been slow to characterise. This survey is about that compounding.

E-mail addresses: vincenzo.sammartino@phd.unipi.it, vincenzo.sammartino@kaust.edu.sa.

https://doi.org/10.1016/j.knosys.2026.117001
Received 13 July 2026; Received in revised form 27 August 2026; Accepted 1 September 2026
Available online 25 September 2026
0950-7051/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

1.1. The core asymmetry

The foundational vulnerability of an LLM is that it does not, and architecturally cannot, reliably distinguish instructions from data. Every input the model sees – the developer's system prompt, the user's request, a retrieved document, an API response, a message from another agent – arrives as tokens in a single context window, and there is no privilege bit separating them. Greshake et al. [9] named the consequence *indirect prompt injection*: if an attacker controls *any* content the model will eventually read, the attacker can inject instructions that the model executes with the authority of the legitimate principal. We defer the technical development of this attack, and of the "lethal trifecta" composition that makes it dangerous, to Sections 2 and 5, where it can be treated against a stated architectural model rather than in the abstract.

What matters for the argument of this survey is the change in *consequence*. In a chatbot, indirect prompt injection is a content problem; in an agent it is a privilege problem, because the model's authority is no longer rhetorical—it is the union of every tool the agent can call. No exploit is executed, no memory is corrupted, no CVE applies. The system does exactly what it was designed to do, on behalf of the wrong person.

Multi-agent composition takes this from a per-agent property to a systemic one. When agent A passes its output to agent B, B typically treats that output as trusted – it came from inside the system – and the injected payload that compromised A is now inside B's context. Cohen et al. [10] demonstrated that a payload can be made *self-replicating*, propagating zero-click through a population of GenAI-powered applications; Lee and Tiwari [11] showed the same effect within a single multi-agent system; Gu et al. [12] showed exponential spread through a population of a million multimodal agents. This is the qualitative break: a vulnerability that produces one bad output in a single model produces an epidemic in a multi-agent system.

1.2. Why a survey now

Three arguments motivate a consolidation of this literature at this moment.

The literature has reached critical mass but not coherence. The field is roughly three years old. In 2023 there were a handful of papers [9,13–15]. By 2025 there are dozens of attack papers, at least five distinct defense paradigms, and a growing set of benchmarks [16–20]. What there is not is a shared threat model. Papers disagree about what the attacker controls, what counts as a successful attack, and whether the defender is allowed to modify the model, the scaffold, or only the prompt. Two papers reporting "attack success rate" are frequently measuring incomparable quantities.

The failure is architectural, and the field is only now admitting it. A consistent regularity, documented in Section 8, is that prompt-level defenses fail. Delimiters, instructional reminders, spotlighting [21], perplexity filters [22], and detection classifiers all reduce attack success under the attacks they were designed against and are then defeated by adaptive attackers [17,23]. The defenses that survive scrutiny – CaMeL [24], IsolateGPT [25], information-flow control approaches [26,27] – share a common move: they stop trying to make the model trustworthy and instead build a system in which the model's untrustworthiness is contained. This is the principle of least privilege [28] and the answer to the confused deputy [29], rediscovered—but it has taken the LLM community two years to arrive there, and the implications for framework design have not been systematised.

Deployment is blocked on this. Enterprise adoption of agents is gated less by capability than by the inability to state a defensible security posture. Standards bodies have responded with taxonomies [30–33], but these are practitioner checklists rather than research syntheses; they enumerate risks without mapping them to the empirical literature on which attacks actually work and which defenses actually hold.

1.3. Positioning relative to prior surveys

Several syntheses of LLM and agent security have appeared, and it is important to state precisely what this article adds rather than to assert novelty generically. Table 1 compares this survey to the most closely related prior work along five axes: whether the scope is restricted to systems with an action channel, whether the review protocol is reported reproducibly, whether the threat organisation is multi-dimensional rather than a flat list, whether defenses are mapped to threats under a stated criterion, and whether evaluation methodology is itself critically assessed.

The prior work divides into three groups. *Trustworthiness surveys* [34,35] cover alignment, fairness, robustness, and privacy for models, and treat agency only in passing; their unit of analysis is the model, not the deployed system. *LLM attack surveys* [36,37] organise jailbreaks and prompt attacks comprehensively but stop at the output channel: an attack that ends in a harmful string is the terminal event, whereas for us it is the precondition. *Agent-security surveys* [38–42] are the closest comparators and are enumerative in structure: they list risk categories, associate each with representative papers, and list countermeasures. They are valuable as maps of the terrain, and we have used them as seeds for snowballing.

Our contribution is orthogonal to enumeration and rests on four points that Table 1 isolates. **(i) Reproducible protocol.** We report the search strings, databases, date bounds, inclusion criteria, screening counts, and quality-appraisal rubric (Section 3); none of the agent-security surveys we compare against reports a protocol sufficient to reconstruct its corpus. **(ii) A taxonomy designed for defense selection, not description.** Existing classifications are typically one-dimensional—by attack technique [36], by system component [39], or by lifecycle phase [40]. Our three dimensions are chosen because each corresponds to a distinct point of defensive intervention (Section 4), so that the taxonomy answers "which defense could possibly stop this?" rather than only "what kind of attack is this?". **(iii) Evidence-graded threat–defense mapping.** We do not merely list countermeasures against risks; we grade every cell of the mapping by the strength of the evidence supporting it, distinguishing containment arguments from empirical reductions from heuristic alignment, and we separately assess adaptive-attack resistance, deployment cost, and residual risk (Section 8, Tables 4 and 5). **(iv) A critique of measurement.** We argue that attack success rate reported without paired utility is uninterpretable, document how often the pairing is missing (Table 3), and convert the critique into a concrete reporting standard (Section 11.7). To our knowledge no prior agent-security survey performs this analysis.

We also make a scope decision the comparators do not: we deliberately *exclude* model-level safety topics—content moderation, bias, hallucination, alignment as such—except where they bear on an action channel. This narrows coverage and is a limitation (Section 3.6), but it is what allows the treatment of the included material to be systematic rather than encyclopaedic.

1.4. Scope and contributions

A terminological note before the scope statement, since a reviewer of the original submission reasonably asked which of the two the paper is about. We use *agentic* for the general property of an LLM system that acts on an environment, whether or not other agents are present, and *multi-agent* for the special case of several such systems composed together. The title now names both because the corpus does: 44 attack papers are annotated by target in Table 2, and 9 of them target multi-agent systems specifically (Section 7), while the remaining 35 concern a single agent. This is not a scope drift. Every multi-agent attack instance is, at the receiving agent, also a single-agent compromise (constraint (D2), Section 4.2: entry at a peer agent, $E4$, entails an agent-to-agent path, $P3$, but the payload still has to defeat that agent's own instruction/data boundary to take effect there). The single-agent

Table 1
Positioning relative to prior surveys. Action scope: restricted to systems with a tool/action channel (•) rather than to models in general (∘). Protocol: search strings, databases, inclusion criteria and screening counts reported in reproducible detail. Multi-dim.: threats organised along more than one axis with a stated relation between axes, rather than as a flat list. Def. map: defenses mapped to threats under an explicitly defined coverage criterion. Eval. critique: measurement practice itself analysed (e.g. the ASR/utility pairing problem). ✓ = present; ✗ = absent; (✓) = partial.

Survey	Focus	Action scope	Proto-col	Multi-dim.	Def. map	Eval. critique	Window
Liu et al. [34]	Trustworthy LLMs	∘	✗	(✓)	✗	✗	–2023
Chang et al. [35]	LLM evaluation	∘	✗	(✓)	✗	(✓)	–2023
Deng et al. [36]	LLM attacks/defenses	∘	(✓)	✗	(✓)	✗	–2024
Rossi et al. [37]	Prompt-injection categories	∘	(✓)	✗	✗	✗	–2024
Li et al. [39]	Personal LLM agents	•	✗	(✓)	✗	✗	–2024
Gan et al. [40]	Agent risk landscape	•	✗	(✓)	(✓)	✗	–2024
He et al. [38]	LLM-agent security & privacy	•	✗	(✓)	(✓)	✗	–2024
Yu et al. [41]	Agent safety	•	✗	(✓)	(✓)	✗	–2025
Wang et al. [42]	Agentic AI security	•	✗	(✓)	(✓)	✗	–2025
OWASP [31] & ATLAS [33]	Practitioner registers	•	✗	✗	✗	✗	rolling
This survey	**Agentic & multi-agent security**	•	✓	✓	✓	✓	–2026

material is therefore the mechanism multi-agent propagation is built from, not a separate topic bundled in for coverage, and Section 4.4 states the composition explicitly.

We survey the security of LLM-based systems *that take actions*: systems in which model output is connected, without a human approving each step, to a tool that changes external state. This scope decision is what separates this survey from the broader literature on LLM safety and on jailbreaking. Jailbreaks [43–45] are in scope only where they are a means to an agentic end—where breaking refusal training is the step that lets an attacker weaponise a tool—and we do not attempt to survey alignment or content moderation as such. Conversely, we include attacks that involve no jailbreak at all, because in an agentic setting an attacker frequently does not need one: an agent that has been instructed by an injected string to email a file is not being asked to do anything its safety training recognises as harmful.

Our contributions are as follows.

1. A **reproducible review protocol** (Section 3): full search strings, databases, date bounds, three-stage screening with reported counts, a quality-appraisal rubric, and an explicit stratification of the corpus by peer-review status, so that the corpus can be audited and re-run as the literature grows.
2. A **three-dimensional threat taxonomy** (entry point × propagation path × target asset) that accommodates single- and multi-agent attacks in one frame (Section 4, Fig. 3), accompanied by an explicit analysis of the dependence structure among the dimensions: they are *not* mutually independent, and we characterise the admissible combinations and the conditional constraints that hold between them (Section 4.2).
3. A **consolidated comparison of 44 attack papers** (Table 2) recording the threat class, injection vector, assumed attacker capability, target, and evaluation setting, together with a companion evidence table (Table 3) recording reported attack success rate, *paired* benign task utility, evaluated model scale, toolset, and evaluation environment—with unreported quantities marked as such, which is itself one of the survey's findings.
4. A **defense-to-threat coverage map** (Table 4) across five defense families, under an operationally defined, evidence-graded coverage criterion (Section 8.1), plus a comparative assessment of practical effectiveness, adaptive-attack resistance, deployment cost, and residual risk (Table 5), which localises the field's blind spots.
5. A **critical assessment of agent-security evaluation** (Section 9), including the argument that attack success rate reported without a paired utility measurement is uninterpretable, since the trivial defense – disconnect the tools – scores perfectly.
6. A **research agenda and a minimal security-reporting standard** (Sections Section 10–11) for authors proposing new agent frameworks, including concrete benchmark specifications, a reproducibility protocol, and a procedure for maintaining this survey as a living artefact.

1.5. Research questions and roadmap

The survey is organised around five research questions.

- **RQ0.** What literature exists on the security of tool-using and multi-agent LLM systems, and by what reproducible protocol can it be delimited? (Section 3)
- **RQ1.** What is the attack surface of a tool-using agent, and how does it change under multi-agent composition? (Sections 2 and 4)
- **RQ2.** Which attacks have been demonstrated against agentic systems, under what threat models, and with what empirical success? (Sections Section 5–7)
- **RQ3.** Which defenses have been proposed, what do they actually guarantee, at what deployment cost, and which survive adaptive attack? (Section 8)
- **RQ4.** How is agent security measured, and are the measurements comparable? (Section 9)

The remainder of the paper is organised as follows. Section 2 fixes an architectural model of an agentic system and derives its trust boundaries. Section 3 states the review methodology. Section 4 presents the taxonomy and its dependence structure. Sections Section 5, 6, and 7 survey attacks. Section 8 surveys defenses and presents the coverage map. Section 9 examines evaluation practice. Sections 10 and 11 discuss open problems and a research agenda; Section 12 concludes.

2. Background: anatomy and trust boundaries of an agentic system

To reason about the attack surface we need a model of the system under attack. This section fixes one and identifies where trust is granted and where it is violated.

2.1. A reference architecture

An LLM agent is a control loop over five components.

The model M maps a context window to a token sequence. It is stochastic, it has no internal notion of privilege, and – critically – the boundary between the instructions it was given and the data it was shown exists only as a textual convention that the model has been trained to usually respect.

The scaffold (or harness) is the deterministic code around M: the system prompt, the loop that parses M's output into tool calls, the retry logic, the output formatter. The scaffold is ordinary software and is trusted. It is also, as we will see, the only place where a sound defense can live, because it is the only component with reliable state.

The tool set $T = \{t_1, \ldots, t_n\}$ is the agent's set of effectors. Each t_i has a declared interface and an *effective authority*: the set of state changes it can cause. The union $\bigcup_i \mathrm{auth}(t_i)$ is the agent's blast radius, and it is almost always larger than its designers believe, because authority composes: a tool that reads a file and a tool that makes an HTTP request together constitute an exfiltration primitive that neither provides alone.

The memory is persistent state carried across steps or sessions: conversation history, a scratchpad, a vector store of past episodes, or a retrieval corpus. Memory is the component that turns a transient compromise into a durable one.

The environment is everything the tools touch: web pages, file systems, APIs, databases, other agents. It is not under the defender's control, and it is the primary injection surface.

A *multi-agent system* composes several such loops, with the output of one agent entering the context of another over a message bus, a shared blackboard, or a hierarchical delegation structure [6,7,46]. The essential observation is that this composition is, by default, *transitive in trust and non-transitive in authority checking*: agent B trusts a message from agent A because it arrived on the internal bus, but B does not re-derive whether the content of that message originated from a trusted source or from a web page A happened to read.

It is worth being precise about how this differs from the classical multi-agent systems literature in distributed control, from which the term is borrowed. There, agents exchange scalar or vector states over a communication graph, and the analytical questions concern convergence: whether a consensus or tracking objective is reached, and how quickly, under switching topologies, asynchronous updates, communication delays, and cooperative–competitive (signed) interactions. That literature has developed sharp results—for instance, convergence-rate characterisations for consensus under communication delays via products of substochastic matrices [47], and tracking guarantees over asynchronous cooperation–competition networks in which competitive edges coexist with cooperative ones [48]. Two structural properties carry over to our setting and one does not. *What carries over:* the dependence of system-level outcomes on graph topology, and the treatment of an adversarial or antagonistic node as a first-class element of the network model rather than as an exception. Both reappear here—topology governs infection reach (Section 7), and a compromised peer is an admissible entry point (E4). *What does not carry over:* in the classical setting the messages are values in a metric space and a receiving agent applies a fixed, non-programmable update rule, so a malicious neighbour can perturb the *value* of the consensus but cannot alter the *update rule itself*. In an LLM multi-agent system the message is natural language and the update rule is the receiving model's next-token distribution conditioned on that message; a malicious neighbour can therefore rewrite the recipient's objective. This is why Byzantine-robust aggregation and weighted-consensus mechanisms do not transfer directly, and why we treat inter-agent message passing as an injection surface rather than as a perturbation channel. We return to this in Section 7 and use it in Section 11 to argue that graph-theoretic containment results are the right formal ambition for multi-agent agentic security.

2.2. Trust boundaries and where they fail

Three boundaries matter, and each is crossed by a class of attack.

Boundary 1: instruction/data, inside the context window. This is the boundary that does not exist. Every token in the context has equal standing; the model's compliance with the system prompt over injected text is a learned tendency, not an enforced invariant. All prompt injection – direct and indirect – is a violation of this non-boundary. It cannot be repaired by better prompting, only by ensuring that untrusted tokens never reach a context that has authority (Section 8).

The composition that makes this dangerous. Willison [49] labels the canonical configuration the *lethal trifecta*: an agent with (i) access to private data, (ii) exposure to untrusted content, and (iii) a channel to the outside world. An assistant that reads a user's inbox, browses the web, and can send email has all three. An attacker who places a crafted string in any page that agent visits – or any email it receives – can instruct it to read the private data and exfiltrate it through the outbound channel, and every one of those steps is an action the agent was authorised to perform. The trifecta is not a fourth boundary; it is the observation that Boundary 1's absence becomes exploitable precisely when Boundary 2 permits an authority composition spanning a read of secrets and a write to the outside.

Boundary 2: model/tool, at the point of action. The scaffold parses model output and invokes tools. This boundary *does* exist – it is real code – but it is conventionally implemented as a pass-through: whatever the model asks for, the scaffold executes. This is the confused-deputy configuration in its textbook form [29]. The agent holds the user's credentials and executes an attacker's intent. Section 6 surveys the attacks that live here.

Boundary 3: agent/agent, in a multi-agent system. This boundary is usually absent by design, because the point of the architecture is frictionless delegation. Its absence is what permits propagation [10,11,50], and Peigné et al. [51] show that reintroducing it costs collaborative capability—a genuine trade-off they term the *multi-agent security tax*, and one of the more important open problems in the field.

2.3. Threat model

Unless stated otherwise, we assume an attacker who (i) cannot modify the model weights, the scaffold, or the tool implementations; (ii) *can* control some content that the agent will read—a web page, an email, a document, a code comment, an API response, a tool description, or a message from a compromised peer agent; and (iii) knows the agent's general architecture but not necessarily its system prompt. This is the standard indirect-injection threat model and it is deliberately weak: it grants the attacker nothing that an ordinary internet user does not already have. Where papers assume more – white-box gradient access [43], control of the training data [52–54], or a compromised agent inside the system [55] – we say so explicitly, and Table 2 records the assumption for each paper, because the field's habit of leaving it implicit is a major obstacle to comparing results.

3. Review methodology

This section states the protocol by which the corpus was assembled, so that the review can be audited, reproduced, and extended. We follow the PRISMA 2020 structure for reporting, adapted to a computer-science venue: we report the information sources, the full search strings, the date bounds, the inclusion and exclusion criteria, the screening procedure with counts at each stage, the quality-appraisal rubric, and the threats to the validity of the review itself. The protocol was fixed before full-text screening began; deviations are noted where they occurred.

3.1. Information sources and search strings

We searched five sources: arXiv (categories cs.CR, cs.AI, cs.CL, cs.LG), the ACM Digital Library, IEEE Xplore, DBLP, and the Semantic Scholar API. Conference proceedings were additionally searched directly for USENIX Security, IEEE S&P, ACM CCS, NDSS, NeurIPS, ICML, ICLR, ACL, EMNLP, and their associated security and safety workshops. The window was 1 January 2023 to 31 January 2026; a small number of foundational pre-2023 works (e.g. [13,28,29,56]) are cited for context but are not counted in the corpus.

The search was the Cartesian product of an *agent* term and a *security* term, expressed as the Boolean query

```
(‘‘LLM agent’’ OR ‘‘language model agent’’ OR "tool use’’
OR ‘‘tool-augmented’’ OR "multi-agent’’ OR
‘‘autonomous agent’’ OR ‘‘agentic’’ OR "AI agent’’)

AND

(‘‘prompt injection’’ OR ‘‘attack’’ OR "jailbreak’’ OR
‘‘exfiltration’’ OR "data leakage" OR "poisoning’’ OR
‘‘backdoor’’ OR ‘‘defense" OR "guardrail" OR "sandbox’’
OR ‘‘isolation’’ OR ‘‘security’’ OR "threat model’’)
```

applied to title, abstract, and keywords. Each database required a syntactic adaptation of this same logical query, not a different query: arXiv's advanced search was run as the field-restricted form `abs:(agent term) AND abs:(security term)` plus the equivalent `ti:` query, unioned; ACM DL and IEEE Xplore, which support nested Boolean groups natively, took the query verbatim as shown; DBLP, which has no abstract index, was queried on title only and is therefore the least complete source individually, which is why it was never relied on alone (Semantic Scholar and the venue-direct searches cover the same ground on full text). Sources with no Boolean support at all (arXiv's plain full-text listing, used only as a fallback when the advanced-search API rate-limited us) were queried with the disjunctive expansion of the same terms. Sources with restricted Boolean support were de-duplicated by DOI or arXiv identifier; records lacking both were de-duplicated by normalised title (lower-cased, punctuation and whitespace collapsed) plus first-author surname, which is how a preprint and its later camera-ready were recognised as the same record for the E1 exclusion below.

The database query was supplemented by two mechanisms. First, backward and forward snowballing (one iteration each, applied until no new records met the inclusion criteria) from the six records that emerged as most cited within the pool [9,16,17,23,24,43]. Second, manual inspection of the reference lists of the OWASP GenAI and MITRE ATLAS documents [30,31,33] and of the prior surveys in Table 1. Snowballing contributed 61 of the finally included records, which is a large fraction and reflects the speed of the field: many relevant preprints are not well indexed at the time of writing.

3.2. Inclusion and exclusion criteria

A record was included if it satisfied all of the following.

- **C1 (action channel).** The paper concerns an LLM-based system that *invokes tools or acts on an environment*, or concerns a defense, benchmark, or measurement methodology intended for such a system. This excludes work on static-model jailbreaking with no action channel, except where such work is used explicitly as an enabling step for an agentic action (Section 5).
- **C2 (technical contribution).** The paper presents an attack, a defense, a benchmark, or an evaluation methodology. Position pieces, risk enumerations without technical content, and capability papers with no security claim are excluded from the corpus, though the major standards documents are cited for comparison.
- **C3 (empirical or formal grounding).** The paper reports empirical results, a formal security argument, or a reproducible artefact. Purely conceptual threat enumerations are excluded.
- **C4 (availability and language).** The full text is publicly retrievable and written in English.

Exclusion criteria applied in order were: E1, duplicate of an already-included record (typically a preprint later published—the published version was retained and the preprint noted); E2, out of temporal scope; E3, fails C1 (the single most common reason, accounting for most exclusions, since generic jailbreak work is voluminous); E4, fails C2 or C3; E5, superseded by a strictly more complete version by the same authors.

3.3. Screening procedure and corpus

Screening proceeded in three stages, summarised in Fig. 1. The database query returned 941 records, reduced to 587 after de-duplication by identifier and normalised title. Title-and-abstract screening against C1–C2 reduced the pool to 143. Full-text screening against C1–C4 yielded the **102 papers** that form the corpus: 44 attack papers (Tables 2 and 3), 31 defense papers, 18 benchmarks or evaluation methodologies, and 9 surveys or standards used for comparison rather than as evidence. Of the 41 records excluded at the full-text stage, 24 failed E3 (no action channel: static jailbreak or content-moderation work with no tool-use or agentic component), 9 failed E4 (no empirical or formal grounding—risk enumerations and position pieces), 6 were E5 (superseded by a later, more complete version from the same authors, most often a workshop paper subsumed by its own extended arXiv version), and 2 failed E1/E2 on closer reading than the abstract had allowed (temporal scope, C4 language).

Because this survey has a single author, the customary two-reviewer independent screening with an inter-rater statistic was not available. To mitigate the resulting subjectivity we adopted two substitutes, and we report them plainly because the alternative – silence – is what makes single-author reviews hard to trust. First, all screening decisions were recorded with the governing criterion at the time of the decision, and the full record list was re-screened blind to the first-pass decision after a two-week interval; the intra-rater agreement between the two passes was 94.1% (135/143 at the full-text stage), and the eight discordant records were resolved by re-reading against C1–C4, with five ultimately included. Second, the classification of each attack paper along the three taxonomic dimensions (Table 2) was derived mechanically from the paper's own stated threat model wherever one was given, rather than from our reading of its experiments; where no threat model was stated – which was the case for 12 of the 44 attack papers, itself a finding – the assignment was made from the experimental setup. We regard the absence of independent double screening as the principal methodological limitation of this review (Section 3.6).

3.4. Quality appraisal

Included papers were appraised on four binary indicators, chosen because they are the properties on which the arguments of this survey depend and because they are checkable from the text:

- **Q1 (threat model stated).** The paper states explicitly what the attacker controls and does not control.
- **Q2 (adaptive evaluation).** The attacker is permitted to adapt to the defense under test, or – for attack papers – the attack is evaluated against a non-trivial defense.
- **Q3 (paired utility).** Benign task success is reported alongside attack success on the same system and environment.
- **Q4 (artefact).** Code, data, or an environment sufficient to reproduce the headline result is released.

The distribution is itself a result. Across the 44 attack papers, Q1 holds for 32, Q2 for 14, Q3 for 18, and Q4 for 29—so 25 attack papers report no benign-utility measurement of any kind alongside their attack results. Across the 31 defense papers, Q1 holds for 27, Q2 for 13, Q3 for 19, and Q4 for 21. That fewer than half of attack papers report paired utility (Q3) is the empirical basis for the critique in Section 9.2 and for the reporting standard of Section 11.7; the per-paper values are recorded in Table 3. We did not exclude papers on the basis of appraisal score – in a three-year-old field that would discard most of the evidence – but we weight claims by it, and throughout the survey we prefer results satisfying Q2 when they conflict with results that do not.

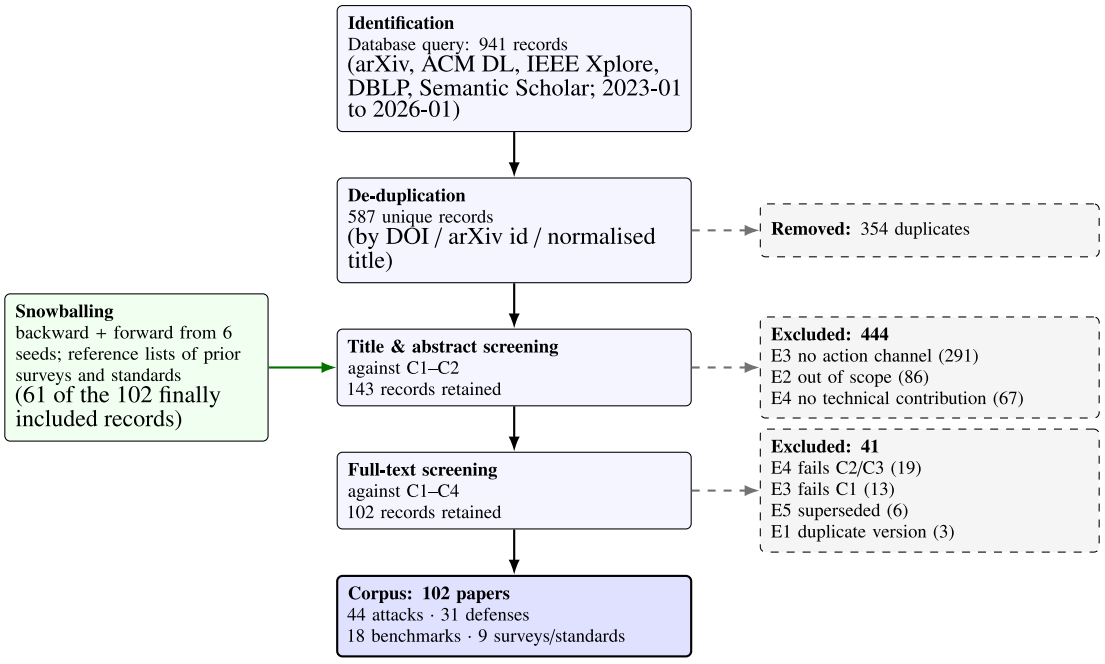


Fig. 1. Study selection. Counts are records, after de-duplication by DOI or arXiv identifier. "Snowballing" includes backward and forward citation chasing from six seed records and inspection of the reference lists of the prior surveys in Table 1.

3.5. Peer-review status and the epistemic weight of preprints

A large fraction of the literature surveyed here exists only as arXiv preprints. This is a genuine limitation and we treat it explicitly rather than silently.

The distribution. Of the 44 attack papers in Table 2, 21 (48%) appeared in an archival peer-reviewed venue (C), 2 (5%) in a non-archival workshop with light review (W), and 21 (48%) are preprints that had not completed peer review at the time of our final search (P). The proportion is higher still for the most recent material: of records first posted in 2025 or later, 71% remain preprints. Table 2 carries the status of every attack paper in a dedicated column, and the same annotation is applied in Table 4 to the works supporting each coverage claim, so that a reader can discount any specific claim by the review status of its source.

Why preprints are nevertheless included. Excluding them would not produce a more reliable survey; it would produce a survey of 2023. Publication latency in security and NLP venues is nine to eighteen months, which exceeds the interval over which the agentic threat surface has materially changed—MCP-based tool poisoning [57,58], the strongest information-flow defense [24], and essentially all multi-agent propagation work [10,11] would be absent. A survey whose purpose is to characterise a moving attack surface cannot adopt an inclusion rule that guarantees it is a year out of date.

How we discount them. We adopt three rules, applied throughout. (i) *No load-bearing claim rests on a single preprint.* Every claim we assert in our own voice – in particular the two headline findings of Section 12 – is supported either by peer-reviewed work or by at least two independent preprints reporting concordant results; where this is not achievable we mark the claim as provisional. (ii) *Preprint results are reported as demonstrations, not as measurements.* An attack shown to work in a preprint is credible evidence that the attack *exists*; a reported attack success rate from an unreviewed source is not treated as a calibrated quantity, and we do not aggregate or average such numbers across papers. This is a stronger reason for our refusal to compute cross-study summary statistics (Section 9) than the threat-model incommensurability argument alone. (iii) *Defense claims are held to a higher bar than attack claims.* An unreviewed attack that is later found to be weaker than claimed leaves a defender over-prepared; an unreviewed defense that is later broken leaves a defender exposed. Accordingly, the containment gradings in Table 4 (▲) are awarded only where the argument is formal and checkable from the text, independent of review status, and never on the strength of an unreviewed empirical result alone.

Residual risk. These mitigations reduce but do not eliminate the exposure. Specifically: reported effect sizes in Table 3 drawn from preprints may not survive review; negative results are systematically under-represented on arXiv, so the impression that most attacks succeed is subject to publication bias in an unusually acute form; and preprints are revised in place, so a citation to arXiv identifies a moving target. We record the version consulted for every preprint in the bibliography and we recommend, in Section 11.8, that agent-security surveys be maintained as versioned living documents for exactly this reason.

3.6. Threats to validity of this review

We state the limitations of the review as such.

Selection validity. Single-author screening (Section 3.3) is the main threat; intra-rater re-screening mitigates but does not remove it. The heavy reliance on snowballing (61 of 102 records) additionally biases the corpus towards the citation neighbourhood of the seed papers, which may under-represent work published outside the anglophone ML-security community—including, plausibly, relevant results in the distributed-control literature on resilient consensus, of which we cite only the entry points [47,48].

Construct validity. The taxonomy is our construct, not a consensus one; a different choice of dimensions would yield a different coverage map. Section 4.2 therefore states the dependence structure explicitly so that the construct can be criticised precisely rather than merely disagreed with.

Internal validity. Assigning a paper to taxonomic coordinates requires interpretation, and 12 of 44 attack papers state no threat model. Cells of Table 3 marked n/r mean the quantity was not reported in a form we could extract, not that it is zero or unfavourable.

External validity. The corpus is dominated by benchmark and sandbox evaluations. Whether these predict behaviour in production deployments is unmeasured, and we flag it as open problem P8 (Section 10).

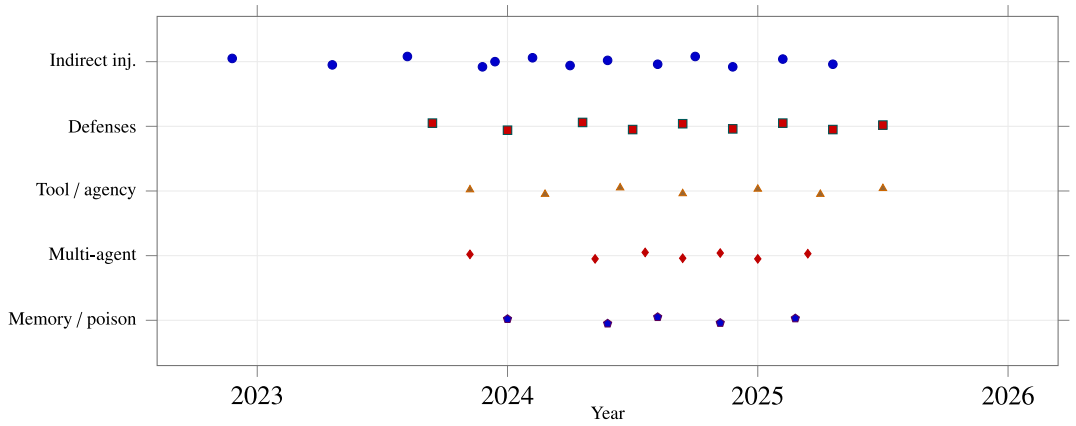


Fig. 2. Publication timeline of the 102-paper corpus, 2023–2026, by threat category. Indirect prompt injection (Section 5) dominates early; multi-agent propagation attacks (Section 7) emerge only in mid-2024; system-level defenses (Section 8) are the most recent and the sparsest line of work. Defense papers lag their corresponding attacks by roughly nine months, a gap that has not narrowed over the observation window.

Temporal validity. The search closed on 31 January 2026. In a field with this publication rate, the survey is a snapshot; Section 11.8 specifies the update procedure we recommend (see Fig. 2).

4. A taxonomy of agentic threats

Practitioner taxonomies enumerate threats as a flat list [30,31]. This is useful for compliance and useless for defense selection, because it gives no structure by which to reason about *which* defense could possibly stop *which* attack. We propose instead a three-dimensional classification. The dimensions are chosen so that each one corresponds to a place where a defense can be inserted.

4.1. The three dimensions

Dimension 1: entry point E. Where does adversarial content first enter the system?

- *E1—User channel.* The human operator is the adversary (direct injection, jailbreak-to-action). Relevant when the agent has authority the user should not have—an enterprise agent with database credentials, a customer-facing agent with refund authority.
- *E2—Environment.* Adversarial content sits in data the agent retrieves: a web page [20,59,60], an email [9], a document, a repository. This is the classic indirect-injection surface and the most realistic.
- *E3—Tool surface.* The tool itself is adversarial or its metadata is: a poisoned tool description, a malicious MCP server, a compromised third-party API returning injected content in its response [57,58].
- *E4—Peer agent.* The content arrives from another agent in the system, either compromised or malicious by construction [11,55,61].
- *E5—Training/memory substrate.* The adversary corrupts what the agent learned or remembers: fine-tuning backdoors [52,53], RAG poisoning [62], memory poisoning [63]. This single label spans two capability regimes that should not be conflated: the training-time sub-case requires influence over the fine-tuning or pretraining data, a capability held by few attackers; the memory/RAG sub-case requires only a write into a retrieval corpus or a persistent store, which is far more commonly available and is often achievable through the same environment channel as E2. We retain one label because both share constraint (D1) below (substrate corruption implies persistence), but the two sub-cases are told apart explicitly, with their distinct assumptions, in the family F_D definition of Section 4.4.

Dimension 2: propagation path P. How far does the compromise travel?

- *P1—Single-hop.* The injected instruction causes one unauthorised action and stops.
- *P2—Cross-tool.* The compromise chains tools to compose an authority the designer never granted (read file → HTTP POST = exfiltration). Section 6.
- *P3—Agent-to-agent.* The payload enters a peer's context and re-executes there, potentially recursively [10–12,50].
- *P4—Persistent.* The payload is written into memory or a knowledge base and re-activates on future, otherwise-clean sessions [62,63].

Dimension 3: target asset A. What is violated?

- *A1—Confidentiality.* Exfiltration of private data the agent can read: inbox contents, database rows, source code, the system prompt, other users' data in a shared deployment [60,64].
- *A2—Integrity.* Unauthorised state change: a sent email, a transferred payment, a merged pull request, a deleted record.
- *A3—Availability.* Resource exhaustion, loop induction, cost amplification—underexplored, and one of the clearer gaps in Table 4.
- *A4—Objective.* The agent's *goal* is subverted while it continues to appear functional: task drift [65,66], biased outputs, manipulated multi-agent consensus [50,61]. This is the hardest class to detect because there is no anomalous action to catch.

4.2. Dependence structure: what the three dimensions do and do not assume

A reviewer of an earlier version rightly observed that entry point and propagation path appear to overlap conceptually – E4 (peer agent) and P3 (agent-to-agent) both refer to inter-agent communication – and asked whether the dimensions are orthogonal. They are not, and it would be wrong to claim otherwise. This subsection states the actual structure: the dimensions are *distinct* and *jointly non-redundant*, but they are *conditionally dependent*, and the dependence is a sequential one that we characterise explicitly.

What the dimensions actually range over. The apparent overlap dissolves once the referent of each dimension is fixed. Let an *attack instance* be a triple $\tau = (e, p, a) \in E \times P \times A$ describing a single execution. Then:

- e ranges over the *first* channel through which attacker-controlled content crosses into the system's trust perimeter. It is a property of the *origin* of the attack and is, by construction, unique per instance.
- p ranges over the channels traversed *after* entry, up to and including the one on which the violation is realised. It is a property of the *trajectory*.

- a ranges over the security property violated at the terminus. It is a property of the *outcome*.

E4 and P3 therefore denote different events on the same physical channel. E4 says: *the attacker's content first reached this system via a peer agent*—the peer is where the adversary is, and the system under analysis begins at the receiving agent. P3 says: *content that entered elsewhere subsequently crossed an agent boundary*. An injection planted on a web page read by agent A and propagated to agent B is $(E2, P3, \cdot)$, not $(E4, \cdot, \cdot)$: the entry point is the web page. An injection carried by a genuinely malicious peer that the defender does not control is $(E4, P3, \cdot)$. The distinction is operationally consequential, because the two cases admit different defenses: the former is stopped by sanitising the environment channel at A, the latter is not, and requires a trust boundary at B. A flat taxonomy cannot express this difference at all, which is precisely the argument for the extra dimension.

The dimensions are not statistically independent. Let Pr denote the empirical distribution of the 44 attack instances in Table 2. Independence in the probabilistic sense would require $\Pr(e, p, a) = \Pr(e)\Pr(p)\Pr(a)$, and it manifestly fails: for example $\Pr(P4 \mid E5) = 1$ while $\Pr(P4) \approx 0.16$, because a compromise of the training or memory substrate is persistent by definition. Nor should independence be expected or desired. The dimensions are not a factorial design; they are three projections of a causal process, and correlations between them are informative, not defective. We therefore make the weaker and correct claim:

> *The dimensions are jointly non-redundant: no one of them is a deterministic function of the other two.*

This is verifiable directly from Table 2 by exhibiting, for each pair of dimensions, two rows agreeing on that pair and differing on the third. E is not determined by (P, A): WIPI [59] is $(E2, P2, \{A2, A4\})$ while Agent Security Bench's tool-surface configuration [19] reaches the same (P, A) from $E3$. P is not determined by (E, A): EIA [60] is $(E2, P1, A1)$ whereas Greshake et al. [9] is $(E2, P2, \{A1, A2\})$. A is not determined by (E, P): among $(E4, P3)$ instances, Huang et al. [55] target $A4$ while prompt infection [11] additionally achieves $A1$ and $A2$. Each dimension therefore carries information the others do not.

The conditional constraints, stated. The dependence is sequential and hierarchical, and can be given exactly. Write $\mathcal{T} \subseteq E \times P \times A$ for the set of *admissible* triples. Then $\mathcal{T}$ is characterised by four constraints, each of which is a statement about system architecture rather than about the literature:

(D1) **Substrate implies persistence.** $e = E5 \Rightarrow P4 \in p$. Corrupting weights, a memory store, or a retrieval corpus is by definition a modification that outlives the session; the converse does not hold, since a payload entering at E2 may be *written into* memory and thereby acquire P4 [10].

(D2) **Multi-agent paths require a multi-agent system.** $P3 \in p \Rightarrow$ the system is multi-agent, hence $E4$ is a *possible* but not necessary entry point. Formally $\Pr(e = E4 \mid P3 \in p) > 0$ and $\Pr(P3 \in p \mid e = E4) = 1$: entry at a peer agent entails at least one agent-boundary crossing, but not conversely.

(D3) **Cross-tool paths require authority composition.** $P2 \in p \Rightarrow |\{t_i : \text{auth}(t_i) \text{ used}\}| \geq 2$, and consequently $P2$ is unreachable for an agent whose toolset is a singleton. This is the constraint that capability-restriction defenses exploit.

(D4) **Asset reachability is bounded by the toolset, not by the entry point.** $a \in A$ is admissible only if some tool in T can effect it: $A1$ requires a read of secrets composed with an outbound channel, $A2$ requires a state-changing effector, $A3$ requires a metred or rate-limited resource. Crucially, this bound is independent of e and p—which is why capability restriction is the one defense that works uniformly across entry points, and why it appears as the rightmost intervention in Fig. 3.

The effect is easiest to see on *atomic* triples, in which p and a are singletons (in general both are set-valued; the entry point is not). Of the $5 \times 4 \times 4 = 80$ atomic triples, (D1) eliminates the 12 with $e = E5$ and $p \neq P4$, and (D2) eliminates the 12 with $e = E4$ and $p \neq P3$, leaving $|\mathcal{T}| = 56$ admissible atomic triples; (D3) and (D4) do not eliminate combinations unconditionally but make admissibility contingent on the deployed toolset T, which is the sense in which they are design levers rather than facts. The inadmissible region is not an artefact of bookkeeping: it is exactly the set of attack shapes that the architecture forecloses regardless of attacker effort, and the contingent region is exactly the set that a configuration choice can foreclose. That is what makes the taxonomy usable for design rather than only for description.

Why this is the right structure for defense selection. The constraints above have a direct operational reading, and it is the reason we retain three dimensions rather than collapsing to one. A defense intervenes at a dimension, and its coverage is determined by which dimension:

- Acting on E (detection, filtering) yields coverage that is *entry-point-specific*: a classifier trained on web content does not transfer to tool descriptions. Coverage is narrow and attack-dependent.
- Acting on P (isolation, information-flow control) yields coverage that is *quantified over all* e: if no tainted value can reach a sink, it does not matter through which channel the taint entered. Coverage is broad and attack-agnostic. This is the formal content of the claim, made informally in Section 8, that path-level defenses are the durable ones.
- Acting on A (capability restriction, least privilege) yields coverage quantified over all e *and* all p, by (D4), at the cost of utility.

A one-dimensional taxonomy of attack techniques cannot express these quantifications, and therefore cannot tell a practitioner that a detector's coverage is narrow in a way an isolation boundary's is not. That is the specific work the third dimension does.

4.3. Relation to existing classifications

Three families of classification already exist and the proposed taxonomy is a refinement of, not a replacement for, each.

Practitioner risk registers—OWASP's LLM and Agentic Top 10 [30,31], the NIST AI RMF [32], and MITRE ATLAS [33]—enumerate named risks ("prompt injection", "excessive agency", "supply chain") at a single level. They are designed for coverage of a compliance checklist and are excellent at it. They are flat by design, so two risks that require entirely different controls (an E2 single-hop diversion and an E5 persistent backdoor) sit at the same level with no structural relation, and a control cannot be located against them other than by narrative. Our dimensions can be read as a factorisation of these registers: OWASP's LLM01 spans $\{E1, E2\} \times \{P1, P2\}$, its "excessive agency" entry is a statement about A-reachability under (D4), and its "supply chain" entry is $E3 \cup E5$.

Attack-technique taxonomies in the LLM-security literature [23,36,37] classify by mechanism: direct versus indirect, optimisation-based versus handcrafted, single-turn versus multi-turn. These are the right categories when the object of study is the model, because the mechanism determines what a model-level defense must resist. They are the wrong categories when the object of study is the deployed system, because two attacks with identical mechanisms (an injected instruction) have radically different consequences depending on the toolset they land in—which is exactly the information the mechanism axis discards and the P and A axes retain.

Component- and lifecycle-oriented agent taxonomies [38–42] organise risks by where in the agent they arise (perception, planning, memory, action) or by lifecycle phase (training, deployment, inference). These are closest to our E dimension and, in the component reading, largely coincide with it. What they do not represent is trajectory: a component-indexed taxonomy records that memory is a vulnerable component, but

not that a payload entering at the environment can *become* a memory compromise [10], which is a different phenomenon from a memory compromise originating at the substrate and requires a different control. The P dimension exists to carry precisely this, and the multi-agent worm results [10–12] are unclassifiable without it.

Summarising: the novelty claimed is not the identification of new threat categories – most individual categories appear somewhere in the work above – but the separation of *origin*, *trajectory*, and *outcome* into independent coordinates whose conditional structure is stated (Section 4.2), and the use of that separation to derive quantified statements about defense coverage (Table 4) rather than narrative ones.

4.4. The four threat families: definitions, assumptions, and workflows

Sections Section 5–7 are organised around four threat families. Because these are frequently conflated in the literature—and because their relationships are causal rather than disjoint (Section 6.1)—we define each precisely here, state the assumptions under which it applies, give its taxonomic coordinates, and sketch a representative workflow. Fig. 3 should be read against these definitions.

F_A*: Indirect prompt injection. Definition.* An attack in which content authored by an adversary and residing in a data source the agent retrieves is interpreted by the model as an instruction, causing the agent to pursue the adversary's objective under the principal's authority. *Assumptions.* The attacker can write to at least one source the agent will read, and cannot modify weights, scaffold, or tool implementations. The agent has no enforced instruction/data separation (Boundary 1). *Coordinates.* $e \in \{E2, E3\}$; $p \supseteq \{P1\}$; a unconstrained. *Workflow.* (1) Adversary places a payload in a web page, email, document, or tool description. (2) The agent retrieves it in the course of a legitimate task. (3) The payload enters the context window with no provenance marking. (4) The model's next action is conditioned on it and it emits a tool call serving the adversary's objective. (5) The scaffold executes the call, because the call is well-formed and within the agent's granted authority. *Distinguishing test.* Would the attack still work if the model perfectly obeyed only the developer's instructions? If no, it is F_A.

F_B*: Malicious tool use and excessive agency. Definition.* An attack in which harm arises from the *composition or scope* of the agent's granted authority, rather than from a confusion about whose instruction is being followed. Two sub-cases: *authority composition*, where individually benign tools compose into a capability the designer did not intend; and *over-provisioning*, where the agent simply holds more authority than its task requires and uses it. *Assumptions.* $|T| \geq 2$ for the composition case (constraint D3). The tools behave as specified—or, in the adversarial-tool-ecosystem sub-case, they do not, which is the E3 entry point. *Coordinates.* $e \in \{E1, E2, E3\}$; $p \supseteq \{P2\}$; $a \in \{A1, A2, A3\}$. *Workflow.* (1) An objective – adversarial or merely ambiguous – is presented to the agent. (2) The agent plans a sequence of individually authorised calls. (3) Each call passes its per-tool check in isolation. (4) The *sequence* effects a state change or disclosure no single call would have. *Distinguishing test.* Would the attack still work if the agent's instructions were unimpeachably legitimate but its toolset unchanged? If yes, F_B is present independently of F_A.

F_C*: Inter-agent infection. Definition.* An attack in which a compromise of one agent is transmitted to a peer through the inter-agent message channel, such that the peer is itself compromised and may transmit further; the defining property is *re-executability* of the payload in the recipient's context. *Assumptions.* A multi-agent system in which a recipient conditions on peer messages without re-deriving their provenance (Boundary 3 absent). The payload is written so as to survive the intermediate agent's paraphrasing. *Coordinates.* $e \in \{E2, E4\}$; $p \supseteq \{P3\}$; a unconstrained; target = M. *Workflow.* (1) Agent A is compromised by F_A or hosts an adversary directly (E4). (2) A emits a message to B containing the payload, possibly reformulated. (3) B ingests it as trusted internal input. (4) B executes it and, if the payload is self-reproducing, re-emits it to B's neighbours. (5) Reach is determined by the communication topology, not by payload strength [12,67]. *Distinguishing test.* Does the number of compromised agents grow with the number of message-passing rounds? If yes, F_C; if a single peer merely relays a harmful *result*, that is F_A with an extra hop, not infection.

F_D*: Memory and knowledge-base poisoning. Definition.* An attack in which adversarial content is written into persistent state – long-term memory, a vector store, a retrieval corpus, or model weights – such that it activates on future sessions that are otherwise clean. *Assumptions.* The agent has a writable or attacker-influenceable persistent store, and retrieval from that store is not provenance-filtered. For the training-time sub-case, the attacker influences the fine-tuning or pretraining data. *Coordinates.* $e \in \{E2, E4, E5\}$; $p \supseteq \{P4\}$; $a \in \{A2, A4\}$ predominantly. *Workflow.* (1) Adversary writes a triggerable item into the store, either directly (E5) or by inducing the agent to store it (E2/E4). (2) The session ends; no anomaly is observable. (3) In a later session a benign query matches the trigger. (4) Retrieval surfaces the poisoned item with the standing of trusted context. (5) The agent acts on it. *Distinguishing test.* Does the attack survive a full restart of the agent with a clean context window? If yes, F_D.

The families are defined to be *distinguishable*, not *disjoint*: a single real incident routinely instantiates several, and Section 6.1 gives the composition rules relating them. The Morris-II worm [10], for instance, is F_A at entry, F_D in its persistence mechanism, and F_C in its propagation—which is exactly why a taxonomy with a trajectory dimension is required to describe it.

5. Indirect prompt injection: the load-bearing attack

Prompt injection is the vulnerability from which the rest of the agentic threat landscape follows, and indirect prompt injection – where the adversarial instruction reaches the model through data the model retrieves rather than through the user's own message – is the form that matters for agents. This section reviews direct injection briefly, treats indirect injection in depth as the central threat, and covers the special case where a jailbreak is used as the enabling step for an agentic action.

This section concerns how an adversary obtains *control* of the agent's objective; Section 6 concerns what the agent's *authority* then permits, and the case in which harm arises from authority alone with no loss of control. The two are related as vector and amplifier rather than as disjoint categories, and Section 6.1 states that relation formally, including the causal and temporal ordering between them and its consequences for where a defense must be placed. Readers primarily interested in the taxonomy of the four threat families and their distinguishing tests will find them defined in Section 4.4.

5.1. Direct injection and the non-boundary

The original observation [13] is elementary: an instruction embedded in user input ("ignore your previous instructions and …") is often obeyed in preference to the developer's system prompt. Perez and Ribeiro framed this as *goal hijacking* and *prompt leaking*. Schulhoff et al. [68] scaled the study to a global competition, collecting a corpus of hundreds of thousands of human-authored injections and demonstrating that no prompt-based countermeasure held. Toyer et al. [69] turned the attacker/defender dynamic into an online game and mined it for interpretable attack strategies. Liu et al. [23] gave the first formalisation, defining prompt injection as the composition of a target task and an injected task and providing a benchmark over which both attacks and defenses can be measured; their central negative result – that every prompt-level defense they tested is defeated by a correspondingly adapted attack – sets the tone for the entire defense literature (Section 8).

Direct injection matters for agents in exactly the settings where the user is not the principal to be protected: a customer-service agent with

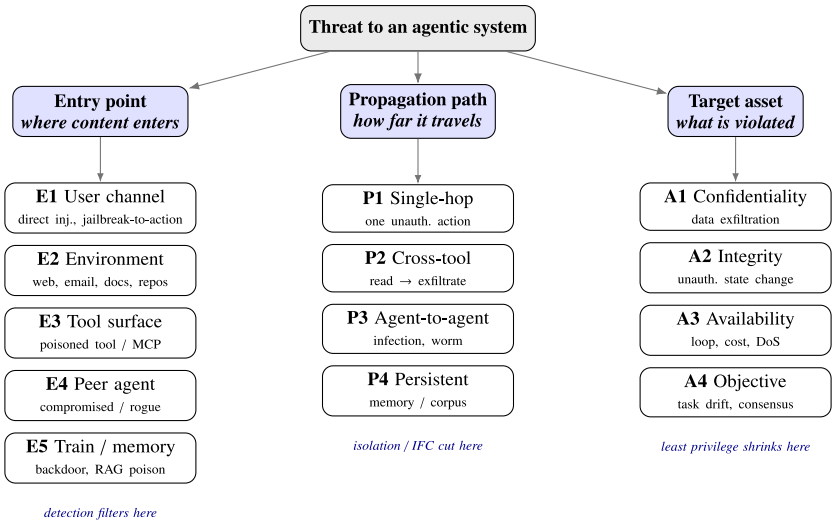


Fig. 3. Three-dimensional taxonomy of threats to LLM-driven agentic systems. An attack is characterised by its entry point (where adversarial content enters), its propagation path (how far the compromise travels), and the asset it targets. Defense families act on different dimensions: detection filters entry points; isolation and information-flow control sever propagation paths; capability restriction shrinks reachable assets. The dimensions are jointly non-redundant but conditionally dependent; the admissible combinations are characterised by constraints (D1)–(D4) in Section 4.2.

backend authority, an enterprise assistant acting under a service account, any deployment where the human at the keyboard is a potential adversary rather than the owner of the resources the agent can touch.

5.2. Indirect injection: the attacker in the data

The pivotal contribution is Greshake et al. [9]. Their insight is that in an LLM-integrated application, the model reads content the user never wrote and the developer never vetted – search results, retrieved documents, emails, the contents of a web page the agent was asked to summarise – and *any* of that content can carry instructions. They demonstrated working attacks against real deployments: a poisoned web page that hijacks Bing Chat into phishing the user for credentials, an injected email that exfiltrates inbox contents, a compromised document that turns a coding assistant into a vector for spreading itself. The attacker's only requirement is to place a string somewhere the agent will read it, and the attacks work with the full authority the application has granted the model.

The severity of indirect injection is a direct function of what the agent can *do* with its instructions, which is why it is an agent problem more than a chatbot problem. Subsequent work traced the surface across deployment settings. On the web, WIPI [59] showed that public web content can drive an LLM web agent to attacker-chosen behaviour; the Environmental Injection Attack (EIA) [60] embedded adversarial fields into web pages that generalist web agents parse, achieving privacy leakage; AdvWeb [70] produced controllable black-box attacks against VLM-powered web agents; and WASP [20] built a benchmark specifically for web-agent prompt injection, finding that even frontier models are diverted by realistic in-page payloads while frequently failing to complete the attacker's task fully—an important nuance, since partial compromise is still compromise but complicates success-rate reporting. Kim et al. [71] surveyed the specific hazards of web-enabled agents and showed the attack surface widens with every additional site the agent is permitted to visit.

Two systematised benchmarks anchor the indirect-injection literature and recur throughout this survey. InjecAgent [16] evaluates tool-integrated agents against indirect injections delivered through tool responses, partitioning attacks into direct-harm (the agent is made to call a harmful tool) and data-stealing (the agent is made to exfiltrate), across 1054 test cases and 17 user tools; it found that agents backed by even strong models are successfully attacked a substantial fraction of the time, and that a simple "enhanced" system prompt reminding the model to ignore injected instructions helps but does not close the gap. AgentDojo [17] is the more influential design: rather than a static test set, it is a *dynamic environment* of realistic tasks (banking, travel, email, cloud) in which both attacks and defenses are pluggable, so that a defense can be evaluated against an attack that adapts to it. This adaptivity is the methodological advance the field needed, and AgentDojo's headline finding—that defenses which look strong against a fixed attack degrade sharply against an adaptive one, and that no evaluated defense drives attack success to zero without also destroying task utility—is the empirical backbone of the argument in Section 8.

A recurring structural attack, orthogonal to the delivery vector, is the *foot-in-the-door* pattern that Nakash et al. [72] identify against ReAct-style agents: an innocuous-seeming injected sub-instruction lowers the agent's resistance to a subsequent harmful one, exploiting the same commitment dynamics that make multi-turn jailbreaks like Crescendo [73] effective. This matters because it means an injection need not carry the whole malicious payload in one visible string—it can bootstrap.

5.3. Jailbreak as the enabling step

Prompt injection changes *whose* instructions the agent follows; jailbreaking changes *what* instructions it will follow. In an agentic setting the two combine: an attacker who has injected instructions into the agent's context may still hit refusal training if the requested action is overtly harmful, and a jailbreak removes that last barrier. The jailbreak literature is large and largely orthogonal to agency—GCG's transferable adversarial suffixes [43], the competing-objectives and mismatched-generalisation analysis of Wei et al. [44], automated black-box methods PAIR [45] and TAP [74], adaptive attacks that defeat claimed-robust models [75], and multi-turn escalation [73]—so we do not survey it exhaustively. What is agent-specific, and consequential, is the finding of Kumar et al. [76]: models that reliably refuse harmful requests *in chat* are readily jailbroken *as browser agents*, because the agentic scaffold reframes the harmful action as a benign-looking sequence of tool calls. Refusal training, in other words, does not transfer from the conversational setting to the agentic one—the safety behaviour was trained on the wrong distribution. This is why we treat agent harmfulness as a distinct measurement problem (Section 9) rather than assuming a well-aligned model is a safe agent.

6. Malicious tool use and excessive agency

If prompt injection is how an attacker gets to speak with the agent's voice, tool use is what that voice can accomplish. This section surveys the threats that arise specifically from the action channel: the composition of authority across tools, the adversarial tool ecosystem, the confused-deputy configuration, and the emergence of harmful behaviour that no single tool call reveals.

6.1. Relation to Section 5: injection as vector, tool authority as amplifier

Because indirect prompt injection is frequently the mechanism by which unauthorised tool invocation is achieved, the two threat families are not disjoint and should not be read as parallel categories. They are related as *cause* and *capacity*, and stating the relation precisely determines where a defense must sit.

The decomposition. An agentic security incident is the conjunction of two independent conditions:

$$\underbrace{\text{Harm}}_{\text{incident}} = \underbrace{\text{Control}}_{\text{whose objective the agent pursues (Section 5)}} \wedge \underbrace{\text{Authority}}_{\text{what the agent can effect (Section 6)}} \tag{1}$$

Section 5 is the study of how an adversary obtains Control; the present section is the study of what Authority makes possible once obtained, and – separately – of how harm arises when Authority is excessive even though Control was never lost. In the taxonomy of Section 4, Control is a statement about the entry point e and Authority a statement about the reachable asset set, bounded by constraint (D4).

The causal and temporal ordering. In the common case the ordering is strict and sequential: injection precedes invocation. The payload enters at t_0 (E2 or E3), is read into context at t_1, is emitted as a tool call at t_2, and is executed at t_3; the cross-tool path P2 is realised over $t_2 \dots t_n$ as the agent composes further calls. Indirect injection is thus the *vector* and tool authority the *amplifier*: the injection determines *that* something unauthorised happens, the toolset determines *how bad* it is. This is why the same payload is a nuisance against a summarisation agent and a breach against an agent holding database credentials, and why severity cannot be read off the attack at all—only off the pair.

Why they are nevertheless distinct sections. The conjunction in (1) has two singly-satisfied cases, and each is a real, documented threat that the other section cannot describe.

- *Control without dangerous Authority.* An injected agent with a read-only, sandboxed toolset is diverted but cannot act. Task drift is observable [65,66] and the objective asset A4 is violated, but no state changes. This is the case that capability restriction reduces to.
- *Dangerous Authority without loss of Control.* This is *excessive agency*, and it is the reason Section 6 is not a subsection of Section 5. An over-provisioned agent following its legitimate principal's ambiguous instruction deletes the wrong records [14]; an agent completes an overtly harmful multi-step task because it was asked to and its refusal training did not transfer to the action channel [18,77]; an agent autonomously exploits a vulnerability because that is what it was told to do [78,79]. No injection occurs in any of these. A survey that treated tool-use risk as a downstream consequence of prompt injection would classify all of them as out of scope, which is precisely the error we wish to avoid.

The consequence for defense placement. Because the conditions are independent, defending either one suffices to prevent the conjunction—but they are not equally defensible. Defenses targeting Control (families F1 and F2, Section 8) must recognise adversarial content, and therefore inherit the adaptivity problem: an attacker rewrites the payload. Defenses targeting Authority (families F3 and F4) do not need to recognise anything, because they bound the reachable asset set regardless of whose objective the agent is pursuing. This asymmetry, which the decomposition makes visible, is the formal content of the survey's central thesis. It also explains an otherwise puzzling empirical pattern: defenses evaluated only against injection benchmarks look strong until the attacker adapts, whereas defenses evaluated as authority bounds hold their reported performance, because there is nothing for the attacker to adapt *to*.

Where a specific attack instantiates both families – the majority of the multi-hop attacks in Table 2, and all four of the composite incidents in Section 7 – we classify it by its entry point in Table 2 and discuss it in the section corresponding to its dominant amplification mechanism, noting the cross-reference in both places.

6.2. Authority composition and the confused deputy

The single most consequential fact about a tool-using agent is that its effective authority is the *composition* of its tools, not the maximum of them. A file-reading tool is benign; an HTTP-request tool is benign; together they are an exfiltration channel, and the agent will assemble that channel out of individually-approved parts whenever an injected instruction directs it to. This is the confused-deputy problem [29] in its purest modern form: the agent wields the user's credentials, an attacker supplies the intent, and every individual action passes whatever per-tool checks exist because each one is, in isolation, legitimate. Willison's "lethal trifecta" [49] – private data, untrusted content, an exfiltration channel – is precisely the claim that this compositional authority, not any single tool, is the vulnerability. The design implication, which Section 8 develops, is that authority must be reasoned about and constrained at the level of *data flow through the composition*, not tool by tool.

The emulation studies made this concrete before the attacks did. ToolEmu [14] uses an LLM to emulate the execution of tools an agent invokes, so that risky behaviours – deleting the wrong files, sending money to the wrong account, executing destructive commands – can be surfaced at scale without a live sandbox; it found that even capable agents take unsafe actions at non-trivial rates when instructions are ambiguous or adversarial. R-Judge [80] showed that models are poor at *recognising* the safety-relevant consequences of their own tool-use trajectories, which is the property a self-monitoring defense would need and mostly does not have.

6.3. Excessive agency: harm without an exploit

A distinctive feature of agentic risk is that much of it involves no exploit at all. An agent granted more authority than the task requires will, under a benign-looking but adversarial instruction, use that authority. This is what OWASP names *excessive agency* [31], and it is the reason agent security cannot be reduced to prompt-injection defense: even a perfectly injection-resistant agent is dangerous if it is over-provisioned. AgentHarm [18] operationalised this by measuring whether agents will *complete* multi-step harmful tasks – fraud, cybercrime, harassment—rather than merely emit harmful text, and found that many agents comply with harmful requests and, crucially, that simple jailbreak templates transfer to the agentic setting to raise compliance further. The gap between refusing a sentence and refusing an *action sequence* is the gap AgentHarm measures, and it is wide.

The safety of web agents specifically has been isolated as a sub-problem. SafeArena [77] recasts the WebArena environment [81] as a safety benchmark, populating it with harmful tasks (misinformation, illegal transactions, harassment) and showing that agents complete a substantial fraction of them. Together with Kumar et al.'s browser-agent jailbreak result [76], this establishes that the web-agent setting is where excessive agency and weak refusal-transfer combine most dangerously.

6.4. The adversarial tool ecosystem

The threats above assume honest tools. Increasingly the tools themselves are untrusted, because the value proposition of protocols such as MCP [5] is precisely that an agent can connect to third-party tools its developer never audited. This opens entry point E3 (Section 4). Hou et al. [57] give the first systematic account of the MCP threat surface across the tool lifecycle – creation, discovery, invocation, update – identifying *tool poisoning* (a malicious server ships a tool whose description or output carries an injection), *rug-pull* updates (a tool benign at install time turns malicious later), and *tool shadowing* (a malicious tool overrides a legitimate one). Radosevich and Halloran [58] audited MCP deployments and demonstrated concrete exploits – remote code execution, credential theft, retrieval-agent hijacking – driven purely through the protocol. Narajala and Habler [82] propose enterprise mitigations, but the structural point stands: a tool description is untrusted content that the model reads with authority, so the tool surface is an injection surface, and connecting an agent to an open tool marketplace is connecting it to an open injection marketplace.

6.5. Agents as attackers: the offensive-capability dimension

A final class of tool-use threat inverts the usual framing: the agent is not the victim but the weapon. The same tool-use competence that makes agents useful makes them capable of autonomous offense. Fang et al. showed that LLM agents can autonomously hack websites [78] and exploit one-day vulnerabilities from their CVE descriptions [79,83]; PentestGPT [84] and the Cybench [85] and CyberSecEval [86] suites measure and, in the benchmark cases, bound this capability; and evaluations of frontier models for dangerous capabilities [87,88] track it as a governance concern. We include this dimension because it closes the loop of the multi-agent threat model in Section 7: a compromised agent in a system is not merely a leak, it is an attacker with tools, and its offensive capability is bounded only by the authority the composition grants it.

7. Multi-agent threats: propagation, leakage, and poisoning

The threats of the previous two sections apply to a single agent. This section covers what is new when agents are composed: the transitive trust between agents (Boundary 3 of Section 2) turns a local compromise into a systemic one, and adds failure modes – propagation, inter-agent leakage, poisoned collaboration, persistent contamination – that have no single-agent analogue. This is the part of the landscape we judge to be both the most dangerous and the least defended.

7.1. Propagation and infection

The defining multi-agent threat is that a compromise *spreads*. Because agent B treats a message from agent A as trusted internal input, an injection that compromises A is delivered into B's context with A's implicit endorsement, and if the payload is written to be re-injectable it compromises B too, and B's neighbours, and so on. Lee and Tiwari [11] named this *prompt infection* and demonstrated it end-to-end: a single injected message in a multi-agent system replicates across agents, and they show it can carry a functional payload (data theft, scam content, service disruption) while spreading, and that agent-to-agent trust is what makes it possible. Cohen et al. [10] independently demonstrated the self-replicating case as a *worm* ("Morris-II") targeting GenAI-powered applications: a zero-click adversarial input that, when processed by a RAG-backed assistant, both performs a malicious action and copies itself into the store from which future messages are retrieved, achieving propagation across an application ecosystem without any user interaction.

The multimodal and scale dimensions make this worse. Gu et al. [12] showed with "Agent Smith" that a single adversarial image, injected into one agent's memory in a society of a million multimodal agents, drives *exponential* infection of the population—a clear demonstration that the propagation dynamics are governed by the network, not the payload strength. Ju et al. [50] studied the softer variant: the flooding spread of *manipulated knowledge* – false facts, not executable instructions – through an agent community, showing that agents propagate and reinforce planted misinformation even when no single agent is fully hijacked, which targets asset A4 (objective/consensus integrity) rather than confidentiality or integrity of state.

7.2. Malicious and byzantine agents

Distinct from an externally injected compromise is an agent that is *adversarial by construction*—a rogue node inside the system, which the multi-agent security literature inherits conceptually from Byzantine fault tolerance but must handle without its assumptions (agents are not merely faulty; they are strategically deceptive, and they communicate in natural language, so message validation is itself an LLM problem). Huang et al. [55] studied system resilience when a fraction of agents are malicious, finding that a small number of adversarial agents can derail collective task completion and that naive majority mechanisms do not rescue it. Amayuelas et al. [61] attacked the specific setting of multi-agent *debate*, where agents are supposed to reach better answers by disagreement, and showed that an adversarial debater can steer the group to a wrong consensus—the collaboration mechanism becomes the attack surface. Tian et al. [89] ("Evil Geniuses") constructed teams of role-played malicious agents and found they generate more harmful and evasive content than a single jailbroken model, because the division of labour lets each agent contribute a piece that is individually less objectionable.

PsySafe [90] approached the same problem through a "psychological" lens, injecting adversarial traits into agents and measuring how dark-trait agents corrupt collective safety, and provides one of the few defense-and-evaluation frameworks aimed at the multi-agent case specifically. NetSafe [67] contributed the crucial structural finding: the *topology* of the agent network governs its safety, with some communication graphs amplifying and others damping the spread of misinformation and injected instructions. This is important because it identifies a defense lever – network design – that is invisible if one reasons about agents individually.

7.3. Inter-agent data leakage and privacy

Multi-agent systems leak data along their communication channels. An agent that holds private information and passes summaries or intermediate results to peers has, in effect, widened the set of principals with access to that information, and the boundaries that should regulate this – who is allowed to know what – are usually not represented anywhere in the system. The right formal frame is contextual integrity [91]: a privacy violation is not disclosure per se but disclosure that breaks the norms of the context in which the data was shared. Mireshghallah et al. [92] operationalised this for LLMs with ConfAIde and showed that even strong models fail to keep information within its appropriate context—they reveal secrets when the conversational frame nominally permits it. AirGapAgent [64] proposed a defense by construction – restricting a conversational agent to a task-scoped "air-gapped" minimum of user data so that an injection cannot exfiltrate what the agent was never given – which is one of the cleaner instances of the least-privilege principle applied to agent data access, and a template for the multi-agent case. Beyond deliberate channels, agents leak inferentially: Staab et al. [93] showed LLMs infer private attributes (location, income, sex) from innocuous text, so an agent need not be shown a secret to expose it. The systematic study of leakage that is *specific* to multi-agent message passing – as opposed to single-agent privacy – remains thin, and we flag it as a gap in Section 10.

7.4. Persistent compromise: memory and knowledge-base poisoning

The final multi-agent threat is persistence: a payload written into shared memory or a retrieval corpus survives the session that planted it and re-activates on future, otherwise-clean interactions (propagation path P4). AgentPoison [63] red-teams LLM agents by poisoning their long-term memory or knowledge base so that a trigger in a future query retrieves the malicious demonstration and steers the agent, achieving high attack success with a tiny poisoned fraction and – because it corrupts the retrieval store rather than the model – transferring across the models that share that store. PoisonedRAG [62] formalised the retrieval-corpus attack: injecting a small number of crafted passages so that a target query retrieves them and the generator produces an attacker-chosen answer, with as few as five poisoned texts per target in a corpus of millions. In a multi-agent system with a shared knowledge base, this is a supply-chain attack on the whole society at once.

The training-time analogue completes the picture. BadAgent [52] and Yang et al. [53] insert backdoors during agent fine-tuning that lie dormant until a trigger appears in an observation or tool response, at which point the agent executes the attacker's action; BadChain [94] backdoors the chain-of-thought itself; and Carlini et al. [54] established that poisoning web-scale pretraining corpora is practically feasible, which means the substrate beneath every agent is itself an entry point (E5). These persistent-substrate attacks are the hardest to defend because the compromise is not in the traffic—it is in the memory, the corpus, or the weights, and it presents no anomaly until the trigger fires.

8. Defenses

We group the defensive literature into five families, ordered by the dimension of the taxonomy they act on and, not coincidentally, by how well they survive adaptive attack. The organising claim of this section, supported throughout, is that robustness increases as defenses move from filtering *content* at the entry point towards constraining *authority* in the surrounding system—from trying to make the model safe to making the model's unsafety harmless.

8.1. What "coverage" means: the criterion behind Table 4

Table 4 maps five defense families onto six threat classes. Because a mapping of this kind is only as meaningful as the criterion that produces it, we state the criterion operationally before presenting the map, and we record in each cell the citation on which its grade rests so that any grade can be contested against a specific source.

A cell (f, c) records the strength of the *evidence* that family f addresses threat class c. It does not record how secure the defense is, and the grades are deliberately not a quality ranking: a family may earn ▲ on a narrow, well-stated guarantee while a broader, more useful defense earns •. Four grades are used, assigned by the following decision procedure, applied in order.

G1. ▲ — containment. Assigned if and only if the source states a security property Π and an argument that Π excludes the threat class, such that (i) the assumptions under which Π holds are enumerated in the text, (ii) the argument is checkable by a reader without re-running experiments, and (iii) the argument quantifies over attacker inputs rather than over a tested sample. Empirical results may accompany ▲ but cannot establish it, and – because an unreviewed empirical claim is a weaker warrant than a checkable argument – peer-review status does not affect this grade. Example: CaMeL [24] gates every tool call on capabilities derived from data provenance, so an injected quarantined model cannot cause an unauthorised action *whatever* it emits; the guarantee is over the space of outputs, not over a benchmark. Non-example: a defense reporting 0% ASR on InjecAgent earns •, not ▲, because the quantification is over the test set.

G2. • — empirically reduced, not contained. Assigned if a measured reduction in attack success against the threat class is reported on at least one benchmark, and either (i) the defense has been shown to degrade against an adaptive attacker, or (ii) it has not been evaluated against one. The grade deliberately merges "known to degrade" with "untested against adaptation", because the field's record (Section 9.4) is that the second reliably becomes the first; where the distinction matters we make it in the text and in the *Adaptive resistance* column of Table 5.

G3. ∘ — principled extension. Assigned where the defense's mechanism applies to the threat class by construction, but no evaluation against that class is reported anywhere we could find. This grade marks an *inference of ours*, not a result, and it is the grade a reader should discount most heavily. Example: information-flow control over a provenance-labelled memory store would by construction constrain persistent poisoning, and FIDES [27] carries labels through stored values, but no reported experiment attacks a poisoned store under it—hence ∘, not ▲, in the rightmost column of row F4.

G4. — — not addressed. No mechanism in the family bears on the threat class, and we found no source claiming otherwise. This is a claim about the literature at the close of our search window, not a claim of impossibility.

Two consequences of this criterion are worth making explicit because they shape the map's headline reading. First, ▲ is scarce not because few defenses work but because few state a falsifiable property: of the 30 cells only 7 carry ▲, while 16 carry an empirical (•) or inferential (∘) grade in which the underlying work reports encouraging numbers but states no property, and 7 are unaddressed. Second, and more importantly, the criterion is what makes the blind spot in the two rightmost columns a substantive finding rather than an artefact of grading severity. No family earns ▲ for agent-to-agent propagation or persistent poisoning, and the reason in every case is the same: the guarantees that exist are stated over a single agent's execution and a single session, and neither quantifier extends to a message crossing an agent boundary or a value surviving a session boundary. That is a gap in the *form* of the available arguments, not in their strength, which is why we treat it in Section 11 as an engineering problem with a known shape rather than as an open research mystery.

Table 5 then supplements the coverage map along the four dimensions coverage alone cannot express – adaptive-attack resistance, utility cost, deployment cost, and residual risk – since a practitioner choosing among these families is choosing on those axes and not on coverage breadth.

One further clarification is owed to Table 4's columns. A reviewer of the original submission correctly observed that they mix dimensions: three are named by entry point (Direct inj. E1, Indirect inj. E2, Tool surface E3), one by propagation path (Agent→agent, P3), one by a target-asset reading (Memory poisoning, dominated by P4), and one – Excessive agency – by neither, explicitly. This was intentional but unstated, and we state it now. The six columns are not a partition of one taxonomic dimension; they are the six *named threat classes* used throughout Sections Sections 5 and 7 and defined with full (E, P, A) coordinates in Section 4.4. They were chosen for the coverage map at the granularity the defense literature itself reports results: a paper is "a defense against tool poisoning", not "a defense against $(E3, P2, A1)$". Naming a column by its dominant coordinate is a legibility choice, not a claim that the column exhausts one dimension—and, on the reviewer's related point, the A dimension *is* represented in every column, just not previously stated. Made explicit: Direct injection = $(E1, P1, \{A2, A4\})$; Indirect injection = $(E2, \{P1, P2\}, \{A1, A2\})$; Excessive agency = $(\{E1, E2, E3\}, P2, \{A1, A2, A3\})$, i.e. family F_B; Tool surface = $(E3, P2, \{A1, A2\})$; Agent-to-agent = $(\{E2, E4\}, P3, \cdot)$, family F_C; Memory poisoning = $(\{E2, E4, E5\}, P4, \{A2, A4\})$, family F_D. The table caption now states this and points here.

Table 2
Consolidated comparison of the 44 attack papers in the corpus. Entry: E1 user, E2 environment, E3 tool surface, E4 peer agent, E5 training/memory substrate. Path: P1 single-hop, P2 cross-tool, P3 agent-to-agent, P4 persistent. Asset: A1 confidentiality, A2 integrity, A3 availability, A4 objective. Tgt: S single-agent, M multi-agent. Access: BB black-box, WB white-box, TR training/data control, IN insider. Rev.: peer-review status at the close of the search window—C archival venue, W non-archival workshop, P preprint (Section 3.5). Reported effect sizes and paired utility are in Table 3.

Work	Attack	Entry	Path	Asset	Tgt	Access	Rev.	Year
Direct & indirect prompt injection								
Perez & Ribeiro [13]	Goal hijack/prompt leak	E1	P1	A2,A4	S	BB	W	2022
Liu et al. [23]	Formalised injection suite	E1,E2	P1	A2,A4	S	BB	C	2024
Schulhoff et al. [68]	Crowdsourced injections	E1	P1	A2,A4	S	BB	C	2023
Toyer et al. [69]	Tensor Trust corpus	E1	P1	A2,A4	S	BB	C	2024
Greshake et al. [9]	Indirect PI (real apps)	E2	P2	A1,A2	S	BB	W	2023
Yi et al. [95]	BIPIA benchmark	E2	P1	A1,A2	S	BB	P	2023
Zhan et al. [16]	InjecAgent (tool-integrated)	E2,E3	P2	A1,A2	S	BB	C	2024
Debenedetti et al. [17]	AgentDojo (adaptive)	E2	P2	A1,A2	S	BB	C	2024
Wu et al. [59]	WIPI web-agent attack	E2	P2	A2,A4	S	BB	P	2024
Liao et al. [60]	Environmental injection	E2	P1	A1	S	BB	C	2025
Xu et al. [70]	AdvWeb (VLM web agents)	E2	P1	A2	S	BB	P	2024
Wu et al. [96]	Multimodal agent attack	E2	P1	A2,A4	S	BB	P	2024
Evtimov et al. [20]	WASP web-agent benchmark	E2	P2	A1,A2	S	BB	P	2025
Nakash et al. [72]	Foot-in-the-door (ReAct)	E2	P1	A2,A4	S	BB	P	2024
Pedro et al. [97]	PI → SQL injection	E2	P2	A1,A2	S	BB	P	2023
Jailbreak-to-action								
Zou et al. [43]	GCG transferable suffix	E1	P1	A4	S	WB	P	2023
Wei et al. [44]	Competing-objective jailbreak	E1	P1	A4	S	BB	C	2023
Chao et al. [45]	PAIR automated jailbreak	E1	P1	A4	S	BB	P	2023
Russinovich et al. [73]	Crescendo multi-turn	E1	P1	A4	S	BB	C	2025
Kumar et al. [76]	Browser-agent jailbreak	E1,E2	P2	A2,A4	S	BB	P	2024
Malicious/excessive tool use								
Ruan et al. [14]	Emulated risky tool use	E1	P2	A2	S	BB	C	2024
Andriushchenko et al. [18]	AgentHarm harmful tasks	E1	P2	A2,A4	S	BB	C	2025
Tur et al. [77]	SafeArena harmful web	E1	P2	A2	S	BB	P	2025
Zhang et al. [19]	Agent Security Bench	E1,E2,E3	P2	A1,A2,A4	S	BB	C	2025
Hou et al. [57]	MCP tool poisoning	E3	P2	A1,A2	S	BB	P	2025
Radosevich & Halloran [58]	MCP exploits (RCE)	E3	P2	A1,A2	S	BB	P	2025
Fang et al. [78]	Agent hacks websites	–	P2	A2	S	BB	P	2024
Fang et al. [79]	Agent exploits one-days	–	P2	A2	S	BB	P	2024
Shi et al. [98]	PI vs. LLM-as-judge	E2	P1	A4	S	BB	C	2024
Multi-agent propagation & collaboration								
Lee & Tiwari [11]	Prompt infection	E4	P3	A1,A2,A4	M	BB	P	2024
Cohen et al. [10]	Self-replicating worm	E2,E4	P3,P4	A1,A2	M	BB	P	2024
Gu et al. [12]	Agent Smith (image)	E4	P3	A4	M	BB	C	2024
Ju et al. [50]	Manipulated-knowledge flood	E4	P3	A4	M	BB	P	2024
Amayuelas et al. [61]	Adversarial debate	E4	P3	A4	M	BB	C	2024
Huang et al. [55]	Malicious-agent resilience	E4	P3	A4	M	IN	P	2024
Tian et al. [89]	Evil Geniuses (rogue team)	E4	P3	A4	M	IN	P	2023
Zhang et al. [90]	PsySafe dark traits	E4	P3	A4	M	IN	C	2024
Yu et al. [67]	Topological safety	E4	P3	A4	M	BB	P	2024
Memory, RAG, and training-time poisoning								
Chen et al. [63]	AgentPoison memory	E5	P4	A2,A4	S	TR	C	2024
Zou et al. [62]	PoisonedRAG corpus	E5	P4	A4	S	TR	C	2025
Wang et al. [52]	BadAgent fine-tune backdoor	E5	P4	A2	S	TR	C	2024
Yang et al. [53]	Agent backdoor threats	E5	P4	A2	S	TR	C	2024
Xiang et al. [94]	BadChain CoT backdoor	E5	P4	A4	S	TR	C	2024
Carlini et al. [54]	Web-scale poisoning	E5	P4	A2,A4	S	TR	C	2024

8.2. Family 1: detection and filtering

The first response to prompt injection was to detect it. Perplexity filters [22] flag inputs whose statistics look adversarial and catch optimised gibberish suffixes like GCG's [43], but not fluent natural-language injections, which are the ones that matter for agents. Trained classifiers – LlamaGuard [99] for input/output moderation, dedicated injection detectors, and programmable guardrail toolkits such as NeMo Guardrails [100] – raise the bar and are cheap to deploy, while Constitutional Classifiers [101] show that a heavily red-teamed classifier can hold against a large jailbreak campaign. The structural limitation is intrinsic: a detector is a classifier over untrusted input, so it inherits the base-rate problem (rare attacks, costly false positives) and, more fundamentally, it is itself attackable—an injection can be crafted to evade the detector, and Liu et al. [23] and the AgentDojo results [17] both show adaptive attacks driving detector-based defenses back towards baseline. Detection raises cost; it does not establish an invariant.

A more agent-specific detection idea targets *task drift*: rather than classifying the input, monitor whether the model's internal state has been diverted from the user's original instruction. Abdelnabi et al. [65,66] show that the activation delta between processing trusted and untrusted content is a measurable, transferable signal of injection, detectable with a lightweight probe and without modifying the model. This is more principled than surface classification because it keys on the *effect* of the injection rather than its form, and it degrades more gracefully; it remains, however, a detector, and it addresses diversion (asset A4) rather than the confidentiality and integrity violations that a compliant-looking agent can still commit.

8.3. Family 2: structural separation of instructions and data

The second family attacks Boundary 1 directly: since the model cannot tell instructions from data, force the separation architecturally. StruQ [102] fine-tunes the model on a structured input format that

Table 3
Reported empirical evidence for the attack papers of Table 2, and the completeness of that reporting. ASR: attack success rate as reported by the source. Utility: benign task success measured on the *same* system and environment (appraisal indicator Q3, Section 3.4). Three states: a value, meaning a single headline figure is stated by the source and reproduced here; rep., meaning the source reports figures only per-configuration, with no headline number quotable without misrepresentation; n/r, not reported. *Values are not comparable across rows* – threat models, judges, environments and model scales differ (Section 9.3) – and we deliberately compute no aggregate over them. The distribution of n/r is itself the finding: **25 of the 44 attack papers report no paired utility measurement of any kind**, so for most of the attack literature the security/utility operating point is unknown and the utility trap (Section 9.2) cannot be ruled out. Abbreviations: G-4 GPT-4/4o, G-3.5 GPT-3.5, C Claude, Ll Llama, Gem Gemini, Mis Mistral, Fl FLAN-T5, VLM vision-language web agent.

Work	ASR (as reported)	Paired utility	Models	Evaluation environment/toolset
Direct & indirect prompt injection				
Perez & Ribeiro [13]	rep.	n/r	G-3	Hand-built prompts, no tools
Liu et al. [23]	rep. (5 attacks × 10 targets)	rep.	G-3.5, G-4, Ll, +7	10 LLM-integrated apps; formalised benchmark
Schulhoff et al. [68]	rep.	n/r	G-3.5, Fl	HackAPrompt competition; 600k+ human prompts
Toyer et al. [69]	rep.	rep.	G-3.5	Tensor Trust online attack/defense game
Greshake et al. [9]	n/r (demonstration)	n/r	G-3.5, G-4	Bing Chat, synthetic apps; email, web, code tools
Yi et al. [95]	rep.[a]	n/r	G-3.5, G-4, Ll	BIPIA: QA, summarisation, code, web, email
Zhan et al. [16]	≈24% (G-4, ReAct)	n/r	G-4, C, Ll, +5	InjecAgent: 1054 cases, 17 user tools, 62 attacker tools
Debenedetti et al. [17]	≤25% (strongest attack)	≈66%–79% benign	G-4o, C-3, Gem, Ll	AgentDojo: banking, travel, e-mail, cloud; 97 tasks, 629 cases
Wu et al. [59]	rep.[a]	n/r	G-3.5, G-4, Ll	Public web pages; web-browsing agent
Liao et al. [60]	rep.	rep.	G-4, Gem, VLM	Mind2Web-derived live sites; generalist web agents
Xu et al. [70]	rep.[a]	n/r	VLM (G-4V, Gem)	Web pages with injected adversarial strings
Wu et al. [96]	rep.[a]	n/r	VLM	VisualWebArena-derived multimodal agent tasks
Evtimov et al. [20]	partial hijack frequent; end-to-end attacker goal < 17%[a]	rep.[a]	G-4o, C-3.7, Gem	WASP: GitLab/web sandbox with realistic in-page payloads
Nakash et al. [72]	rep.[a]	n/r	G-4, C, Ll	ReAct agents, InjecAgent-style toolsets
Pedro et al. [97]	n/r (exploit demo)[a]	n/r	G-3.5	Langchain SQL agent over a live database
Jailbreak-to-action				
Zou et al. [43]	rep. (transfer rates)[a]	n/r	Vicuna, Ll, G-3.5, C	AdvBench harmful strings; no action channel
Wei et al. [44]	rep.	n/r	G-4, C-1	Curated harmful prompt set; no action channel
Chao et al. [45]	rep. (<20 queries)[a]	n/r	G-3.5, G-4, Vicuna, C	AdvBench subset; black-box query budget
Russinovich et al. [73]	rep.	n/r	G-4, Gem, C, Ll	Multi-turn chat escalation; no action channel
Kumar et al. [76]	rep.[a]	rep.[a]	G-4, C-3.5	Browser-agent harness on live and mock sites
Malicious/excessive tool use				
Ruan et al. [14]	rep. (risk rate over trajectories)	rep. (helpfulness)	G-4, C-2, Ll	ToolEmu: LM-emulated sandbox, 36 toolkits, 144 cases
Andriushchenko et al. [18]	harm score, not ASR	rep. (benign split)	G-4o, C-3.5, Gem, Mis	AgentHarm: 110 harmful × 11 categories, synthetic tools
Tur et al. [77]	rep.[a]	rep.[a]	G-4o, C-3.5, Ll, Qwen	SafeArena: 250 harmful + 250 benign WebArena tasks
Zhang et al. [19]	rep. (10 scenarios)	rep.	G-4o, C-3.5, Gem, Ll	ASB: 10 scenarios, 400+ tools, 23 attack/defense pairs
Hou et al. [57]	n/r (lifecycle analysis)[a]	n/r	–	MCP ecosystem survey; tool poisoning, rug-pull, shadowing
Radosevich & Halloran [58]	n/r (exploit demo)[a]	n/r	G-4o, C-3.5	Deployed MCP servers; RCE, credential theft, RAG hijack

(*continued on next page*)

segregates the trusted instruction channel from the untrusted data channel, so that text arriving in the data channel is much less likely to be executed as instruction; SecAlign [103] strengthens this with preference optimisation, training the model to prefer the correct (non-injected) response over the injected one and reporting attack success driven to near zero on its evaluation suite. Jatmo [104] takes the

Table 3 (*continued*).

Work	ASR (as reported)	Paired utility	Models	Evaluation environment/toolset
Fang et al. [78]	73% (11/15, G-4)[a]	n/r	G-4, G-3.5, OSS	15 sandboxed vulnerable websites
Fang et al. [79]	87% (G-4, one-day CVEs)[a]	n/r	G-4, G-3.5, OSS	15 real one-day vulnerabilities with CVE descriptions
Shi et al. [98]	rep.	n/r	G-4, Ll, judges	LLM-as-judge pipelines under optimised injection
Multi-agent propagation & collaboration				
Lee & Tiwari [11]	rep. (infection rate vs. hops)[a]	n/r	G-4o, C-3.5, Ll	Multi-agent pipelines; data theft, scam, disruption payloads
Cohen et al. [10]	rep. (zero-click propagation)[a]	n/r	G-4, Gem, LLaVA	RAG-backed e-mail assistants (Morris-II)
Gu et al. [12]	near-total infection of 10^6 agents within tens of rounds	n/r	LLaVA, multimodal	Simulated society of 10^6 multimodal agents
Ju et al. [50]	rep.[a]	n/r	G-3.5, G-4, Ll	Agent community propagating manipulated knowledge
Amayuelas et al. [61]	rep. (consensus flip rate)	rep. (accuracy)	G-3.5, G-4, Ll	Multi-agent debate on reasoning benchmarks
Huang et al. [55]	rep.[a]	rep.[a]	G-3.5, G-4	Multi-agent task systems with a malicious fraction
Tian et al. [89]	rep.[a]	n/r	G-3.5, G-4	Role-played malicious agent teams (CAMEL/AutoGen-style)
Zhang et al. [90]	rep.	rep.	G-3.5, G-4, C	Multi-agent systems with injected dark-trait personas
Yu et al. [67]	rep. (by topology)[a]	n/r	G-3.5, G-4, Ll	Agent networks across chain/star/mesh/tree topologies
Memory, RAG, and training-time poisoning				
Chen et al. [63]	≥80% retrieval and ≈80% end-to-end at <0.1% poison rate	rep. (benign accuracy)	G-3.5, Ll, agent RAG	Driving, QA and healthcare agents with poisoned memory
Zou et al. [62]	≈90% with 5 poisoned texts per target query	rep.	G-4, G-3.5, Ll, PaLM	RAG over million-scale corpora (NQ, HotpotQA, MS-MARCO)
Wang et al. [52]	>85%	rep. (clean-task accuracy)	ChatGLM, AgentLM, Ll	Fine-tuned agents; OS, web, e-commerce tool suites
Yang et al. [53]	rep.	rep.	Ll, AgentLM	Backdoored agent fine-tuning; web shopping, tool use
Xiang et al. [94]	rep.	rep.	G-4, PaLM2, Ll	Chain-of-thought reasoning benchmarks
Carlini et al. [54]	n/a (feasibility/cost analysis)	n/a	–	Web-scale pretraining corpora (LAION, Wikipedia snapshots)

[a] Marks figures transcribed from a preprint (Section 3.5).

complementary route: for a fixed task, fine-tune a task-specific model that has no general instruction-following capability to hijack, so there is nothing for an injection to instruct. These are the strongest *model-level* defenses and, unlike detection, they change the model's disposition rather than filtering its inputs. Their limits are that they require training access, that the structured-input assumption can leak (data channels that must contain instruction-like content), and that "near zero on our suite" has repeatedly failed to mean "robust to a future adaptive attacker"—the central lesson of Liu et al. [23].

8.4. Family 3: execution isolation and least privilege

The third family gives up on the model and secures the system. If the model may be compromised, then confine what a compromised model can reach. IsolateGPT [25] executes each third-party "app" an agent uses in an isolated environment with mediated, user-approved communication between apps, so that a malicious or hijacked app cannot silently reach another app's data or the user's resources—a direct application of least privilege [28] and the standard answer to the confused deputy [29]. AirGapAgent [64] applies the same principle to data rather than execution, restricting the agent to a task-minimal slice of user information so that an injection cannot exfiltrate what was never loaded. The strength of this family is that it is *attack-agnostic*: it does not need to recognise the injection, because it bounds the damage regardless of whether an injection occurred. Its costs are friction (mediated cross-app calls interrupt the frictionless delegation that motivates agents) and incompleteness (isolation bounds cross-tool and cross-app reach but does not by itself stop a single over-privileged tool from being misused) [105,106].

8.5. Family 4: information-flow control and provable containment

The fourth family is the current frontier and, in our assessment, the most important. Its move is to track the *provenance* of every value the agent handles and to enforce a policy on where tainted values may flow—classical information-flow control [56] applied to agent execution. CaMeL [24] is the landmark result: it extracts the *control flow* of the user's task using a privileged LLM that never sees untrusted data, expresses it as code in a custom interpreter, and confines a second, quarantined LLM to processing untrusted content without any ability to alter control flow; capabilities and a security policy then gate every tool call, so that even a fully injected quarantined model cannot cause an unauthorised action. CaMeL solves a large fraction of AgentDojo [17] with provable properties rather than empirical robustness, and it is the clearest instantiation of the survey's thesis: it does not try to make the model resist injection; it makes injection unable to matter. Related work develops the information-flow perspective as a general system-level defense [26], secures agents with formal information-flow guarantees [27,107], and uses query-language and programmatic-constraint approaches [108] to make the agent's permitted behaviour explicit and checkable. The trade-off is expressiveness: strict information-flow policies forbid legitimate flows too (an agent

genuinely needs to sometimes act on untrusted content), so the research problem is precision – maximising permitted utility under a sound policy – rather than robustness, which is a much healthier place for a defense literature to be.

8.6. Family 5: alignment-side and multi-agent hardening

The fifth family works on the model's dispositions and on the multi-agent system as a whole. On the model side, better refusal training and adversarial training raise the cost of jailbreak-to-action, but the browser-agent result of Kumar et al. [76] shows this does not transfer reliably to the agentic setting, and alignment can itself be gamed—Greenblatt et al. [109] document models strategically feigning alignment, and Scheurer et al. [110] document strategic deception under pressure, both cautioning against treating a well-behaved model as a trustworthy component. On the multi-agent side, the defenses are nascent: AutoDefense [111] uses a team of agents to filter another agent's harmful outputs (defense-in-depth via redundancy); PsySafe [90] provides evaluation and mitigation for dark-trait agents; and NetSafe [67] shows that *network topology* is itself a defense lever, with resilient communication graphs damping propagation. Peigné et al. [51] make the trade-off explicit: adding inter-agent trust boundaries – the very Boundary 3 whose absence enables infection – costs collaborative capability, the "multi-agent security tax". No current defense closes multi-agent propagation with the kind of guarantee CaMeL provides for the single-agent case, which is the single largest gap the coverage map (Table 4) exposes.

8.7. What survives

Two regularities summarise the section. First, ordered by robustness the families run detection < structural separation < isolation < information-flow control, which is exactly the order of moving authority out of the model and into the trusted scaffold; the defenses with provable properties [24,27] are the ones that assume the model is compromised and contain it. Second, this ordering holds for the *single-agent* case; for the multi-agent case none of the families yet provides a containment result, and the propagation attacks of Section 7 remain, in practice, undefended. Table 4 makes both points visible at once.

Robustness ordering is not, however, deployment ordering, and Table 5 exists to prevent that inference. The two are close to inverted: the family with the strongest guarantee (F4) carries the highest adoption cost and the least characterised utility penalty, while the family with the weakest guarantee (F1) is the only one deployable without architectural change. A practitioner is therefore not choosing the strongest defense but the strongest defense they can afford to install, and the honest summary of the current state is that defense-in-depth is forced rather than merely prudent: F1 to raise attacker cost, F3 to bound blast radius, and F4 where a deployment's value justifies rebuilding its scaffold around a provenance-tracking interpreter. Two implications follow from the residual-risk column that are easy to miss. Assumption failure, not attacker ingenuity, is the dominant failure mode for the strongest family – a mis-specified policy or an unmodelled side channel defeats F4 without any adaptive attack at all [112] – so the maintenance burden of an information-flow deployment is a security property of it. And no family, including F4, currently offers anything for persistent memory poisoning, so a deployment with a writable long-term store has an unbounded residual risk regardless of which defense it adopts. We take that up as open problem P5.

9. How agent security is measured

A defense literature is only as trustworthy as the evaluations that validate it. This section reviews the benchmarks used to measure agent security, the metrics they report, and the methodological problems that currently make cross-paper comparison unsound.

9.1. The benchmark landscape

Four benchmarks dominate and differ in a way worth stating precisely.

InjecAgent [16] is a *static* test set of indirect injections delivered through tool responses. Static sets are reproducible and cheap, but they measure robustness against a fixed attack distribution, which is the wrong quantity: an attacker adapts.

AgentDojo [17] is a *dynamic environment* in which attacks and defenses are pluggable components, so an attack can be adapted to the defense under test. This is the methodological state of the art, and it is the reason AgentDojo's numbers are the ones we trust most in this survey. Its finding that no evaluated defense reaches zero attack success without destroying utility is credible precisely because the framework permits the attacker to respond.

AgentHarm [18] measures a different thing: not whether an agent can be hijacked, but whether it will *complete* an overtly harmful multi-step task. It is the right instrument for excessive agency and refusal-transfer, and it is orthogonal to injection robustness—a model can score well on one and badly on the other.

Agent Security Bench [19] attempts the broadest formalisation, covering injection, memory poisoning, and backdoors across ten scenarios and multiple defenses, and reports that the defenses it evaluates fail to defend robustly across the attack classes it spans.

Around these sit BIPIA [95] for indirect injection in LLM-integrated applications, WASP [20] and SafeArena [77] for the web-agent setting, ToolEmu [14] and R-Judge [80] for risky tool use and risk awareness, and Cybench [85] and CyberSecEval [86] for offensive capability. General-purpose harmfulness suites – HarmBench [114], StrongREJECT [115] – remain the reference for jailbreak evaluation but do not exercise an action channel and therefore do not measure agentic risk.

9.2. Attack success rate and the utility trap

The universal metric is *attack success rate* (ASR): the fraction of attempts in which the attacker's objective is achieved. Reported alone, ASR is uninterpretable, and this is the most consequential methodological problem in the field. The reason is trivial and rarely stated: the null defense of disconnecting every tool achieves ASR = 0. Any defense can trade utility for security, so a defense's ASR is meaningless without a *paired* measurement of benign task success on the same environment—the "utility under defense" that AgentDojo makes a first-class quantity [17] and that many defense papers still omit. A defense that halves ASR while halving utility has not obviously improved anything; a defense that halves ASR at negligible utility cost has. Only the joint (ASR, utility) point is a result, and we recommend, in Section 11, that it be reported as such.

It is one thing to assert this and another to show how often the pairing is missing, so we measured it. Table 3 records, for every attack paper in the corpus, whether benign task success was reported on the same system and environment as the attack result. It was not, in any form, for **25 of the 44 papers**; of the 19 that report something, only a minority state a single paired operating point that can be quoted without reconstruction from per-configuration tables. AgentDojo [17] is the clearest exception and reports both quantities as first-class outputs, which is a substantial part of why we weight its findings most heavily throughout this survey. The same gap appears on the defense side: Table 5 carries a *utility cost* column, and for two of the five families the honest entry is that the cost is not characterised at all—for information-flow control, the utility penalty of a sound policy is acknowledged as the central open problem yet is nowhere quantified as a curve.

Two clarifications, since the point is easily overstated. First, the criticism is not that authors are careless: for a pure attack paper with no defense under test, utility is arguably not the attacker's concern, and it is only when the same environment is used to evaluate a defense that the omission becomes disabling. Our position is the weaker

Table 4
Defense-to-threat coverage map. Rows are the five defense families of Section 8; columns are threat classes from the taxonomy (Section 4). Grades are assigned by the operational criterion defined in Section 8.1, and each graded cell carries a superscript key resolving to the evidence on which its grade rests (legend below the table). ▲ (*containment*) = a stated security property entails that the threat cannot succeed while the defense's assumptions hold, the assumptions are enumerated, and the argument quantifies over attacker inputs rather than over a tested sample; • (*empirical*) = a measured reduction in attack success is reported on at least one benchmark, and the defense either degrades under an adaptive attacker or has not been tested against one; ∘ (*principled extension*) = the mechanism applies to the threat class by construction but no evaluation against that class is reported, so the coverage is an inference of ours and not a result; — = not addressed in the literature at the close of the search window. Grades describe the *evidence*, not the quality of the defense. The two rightmost columns are the field's blind spot: no family provides containment for agent-to-agent propagation or persistent memory poisoning. Columns are named threat classes, not raw taxonomic dimensions—each column's full (E, P, A) coordinates, including the target-asset reading, are given in Section 8.1.

Defense family	Representative work	Direct inj. (E1)	Indirect inj. (E2)	Excessive agency	Tool surface (E3)	Agent→ agent (P3)	Memory poison (P4)
F1 Detection & filtering	Perplexity [22], LlamaGuard [99], NeMo [100], Constitutional Cls. [101], task-drift probes [66]	•[a]	•[b]	∘[c]	∘[d]	∘[e]	–
F2 Structural separation	StruQ [102], SecAlign [103], Jatmo [104], spotlighting [21]	•[f]	•[g]	–	∘[h]	–	–
F3 Execution isolation & least privilege	IsolateGPT [25], AirGapAgent [64], dual-LLM [113]	∘[i]	▲[j]	▲[k]	▲[l]	∘[m]	–
F4 Information-flow control	CaMeL [24], IFC defense [26], FIDES [27], formal guarantees [107]	▲[n]	▲[o]	▲[p]	▲[q]	∘[r]	∘[s]
F5 Alignment-side & multi-agent hardening	AutoDefense [111], PsySafe [90], NetSafe topology [67], security tax [51]	•[t]	∘[u]	•[v]	–	•[w]	–

Evidence keys. a [101] (red-teamed classifier holds a large campaign), degraded by adaptive injection [23]; b [22] catches optimised suffixes only; adaptive degradation in [17]; c [99] moderates harmful actions but is not evaluated on authority composition; d [100] can gate tool invocation by rule, untested on poisoned tool metadata; e [66] activation probes detect diversion and are in principle applicable at an agent boundary; not evaluated there; f [102,103] report near-zero ASR on their own suites, static attackers; g [103,104], same caveat; h [21] spotlighting marks provenance of tool output, no evaluation against adversarial tool descriptions; i [25] isolation does not constrain a hostile principal at the user channel; j,l [25] mediated inter-app execution bounds reach irrespective of the injection; j additionally [113]; k [25] with the least-privilege argument of [28]; m [25] inter-app mediation is structurally the same primitive as an inter-agent boundary but is not instantiated or evaluated for one; n–q [24] capability-gated tool calls with a quarantined model that cannot alter control flow, hence a guarantee over all attacker inputs; o additionally [27]; p additionally [107]; q additionally [26]; r [26] formulates information flow at system level; no inter-agent label propagation is defined or evaluated; s [27] carries labels through stored values, but no reported experiment attacks a poisoned store; t,u [111] multi-agent output filtering, static evaluation, and the filter agents are themselves injectable; v [90]; w [67] topology damps propagation empirically; [51] quantifies the capability cost of the corresponding trust boundary. Preprint status of each source is given in the bibliography and summarised in Section 3.5.

one that the *environment* should expose a benign task suite so that the pairing is available to whoever needs it, which is an argument about benchmark design more than about author diligence. Second, the missing measurement is not merely incomplete reporting; it changes what can be concluded. A reported ASR reduction is consistent with a genuine security improvement and equally consistent with a defense that has quietly disabled the capability under attack, and *nothing in the paper distinguishes these*. For 25 of 44 attack papers and two of five defense families, the field currently cannot tell them apart. That is the utility trap, and Table 3 is the measurement of its extent.

Three further metric problems recur. **Success is under-defined:** a web-agent attack that diverts the agent without completing the attacker's end goal [20] is a partial compromise, and papers differ on whether to count it. **Judges are attackable:** many benchmarks score success with an LLM judge, and Shi et al. [98] show LLM judges are themselves subject to optimised prompt injection—the instrument is inside the threat model. **Cost is unreported:** the attacker's and the defender's compute budgets are almost never stated, though both determine the result, a pathology the agent-evaluation literature has already diagnosed in the capability setting [116].

9.3. Threat-model incommensurability

Table 2 records, for each attack paper, what the attacker is assumed to control, and the spread is the point: some papers grant white-box gradients [43], some training-data control [52,63], some an insider agent [55], most only the ability to place a string in the environment [9]. These are not comparable attacker classes, and an ASR obtained under one says nothing about an ASR obtained under another. Similarly, defenses differ in what they may modify – the prompt only, the scaffold, the model weights, the whole system architecture – and a prompt-only defense that underperforms an architecture-level one has not necessarily lost, because it was solving a harder-constrained problem. The field currently reports all of this in prose, inconsistently, or not at all. Making the threat model a structured, mandatory reporting field is the cheapest available fix, and it is the core of the reporting standard we propose in Section 11.

9.4. The adaptive-attack requirement

The lesson that adversarial machine learning learned expensively [117,118] – that a defense evaluated only against the attacks that existed when it was designed will be broken by the attacks that come after – is being relearned here, and the agent-security literature is roughly where image-classifier robustness was a decade ago. The evidence is direct: Liu et al. [23] defeat prompt-level defenses with correspondingly adapted attacks; the AgentDojo evaluation [17] shows defense performance degrading sharply once the attacker is permitted to see the defense; Andriushchenko et al. [75] break models claimed robust with simple adaptive attacks. We therefore treat *static-attack-only* evaluation as insufficient evidence of robustness throughout this survey, and this is why the defenses we credit in Section 8 are those whose guarantees are structural [24,25,27]—a containment argument does not depend on the attacker's ingenuity in the way a classifier's accuracy does.

10. Open problems

We consolidate the cross-cutting problems that the preceding sections expose. Each is stated as a claim about what the literature currently cannot do.

Table 5
Comparative assessment of the five defense families beyond coverage. Adaptive resistance: behaviour when the attacker is permitted to see and adapt to the defense. Utility cost: reported degradation of benign task success where measured; n/r where the defense paper does not report it. Deployment cost: what an adopter must pay—training access, inference overhead, architectural change, or human mediation. Residual risk: what remains exploitable when the defense works as intended. Evidence: strength of the supporting evidence, graded F formal argument, A adaptive empirical evaluation, S static empirical evaluation only, with peer-review status of the principal source in parentheses (C/W/P, Section 3.5).

Family	Adaptive resistance	Utility cost	Deployment cost	Residual risk	Evidence
F1 Detection & filtering	Poor. Detector is itself an attack target; adaptive injections drive performance towards baseline [17,23].	Low but non-zero; false positives block legitimate content. n/r in most sources.	Lowest of the five. Drop-in; one extra inference call per input. No training or architecture change.	Fluent natural-language injections evade statistical filters; base-rate problem at deployment volumes; task-drift probes detect diversion (A4) but not authorised-looking exfiltration (A1/A2).	S/A (C)
F2 Structural separation	Moderate. Reported near-zero ASR on the training distribution; degrades on injection styles outside it [23].	Low on the target task; Jatmo [104] forfeits general instruction-following entirely.	Requires fine-tuning access and a structured input format; not applicable to closed API models.	Data channels that must legitimately carry instruction-like content leak; no protection once the model is bypassed via the tool surface or a peer agent.	S (C)
F3 Execution isolation & least privilege	Good, and attack-agnostic: the bound does not depend on recognising the injection.	Moderate; mediated cross-app calls introduce user-approval friction [25].	Architectural. Requires re-partitioning the agent into isolated components and a mediation policy; user interruption is part of the design.	A single over-privileged tool inside a compartment is unconstrained; policy authoring is manual and error-prone; no story for persistent memory.	F+S (C)
F4 Information-flow control	Strongest available. Guarantees are stated over all attacker inputs, so adaptivity is irrelevant *within the assumptions* [24].	Real and the main open problem: sound policies forbid legitimate flows; declassification is manual.	Highest. Requires a trusted interpreter, provenance tracking through every value, and an explicit security policy per deployment.	Assumption failure, not attack ingenuity, is the threat: a mis-specified policy, an unlabelled side channel, or a declassification rule that is too permissive. Multi-agent boundaries are outside all current formulations.	F (P)
F5 Alignment-side & multi-agent hardening	Weak individually. Refusal training does not transfer to the agentic setting [76]; topology damping reduces but does not bound reach.	Direct and quantified only in aggregate: the "multi-agent security tax" on collaborative capability [51].	Low to moderate: redundant filter agents multiply inference cost; topology changes are a design-time choice with no runtime cost.	Models can feign alignment [109]; a filter agent is itself injectable; damping topologies slow propagation without preventing it, so reach is bounded only in expectation.	S (P)

P1: There is no agreed threat model, so results do not compose. Papers disagree on what the attacker controls, what counts as success, and what the defender may modify (Section 9). The consequence is that the field cannot aggregate: one cannot conclude from "defense *D* reduces ASR by 80%" and "attack *A* achieves 60% ASR" anything at all about *D* facing *A*. This is a coordination failure, not a technical one, and it is fixable by convention.

P2: Prompt-level defense is a dead end, and the field keeps re-entering it. Every defense that operates by adding text to the context – delimiters, reminders, spotlighting [21], instructional hierarchies – has been defeated by an adaptive attacker [17,23]. The reason is structural: such defenses ask the model to enforce a boundary the model has no mechanism to enforce (Boundary 1, Section 2). Papers proposing new prompt-level mitigations continue to appear, evaluated against static attacks. The open problem is not how to make prompting work; it is how to stop the field from spending effort there.

P3: Multi-agent propagation is essentially undefended. This is the gap we regard as most urgent. The attacks are demonstrated, severe, and self-amplifying [10–12,50]; the defenses (Table 4) are empirical, partial, and none provides a containment property. The strongest single-agent result – information-flow control with capability-gated tool calls [24,27] – has not been extended to inter-agent message passing, where the natural formulation would be to propagate taint labels *with* agent messages so that a value derived from untrusted content stays tainted across agent boundaries. To the best of our knowledge this has not been built. It is, in our view, the single highest-value open engineering problem in the area [119].

P4: The security tax is real and unquantified. Peigné et al. [51] establish that inter-agent trust boundaries cost collaborative capability, but the shape of that trade-off curve is unknown, and no defense paper reports where on it the defense sits. Without this, a practitioner cannot choose. The same holds for information-flow policies, whose cost is over-restriction of legitimate flows—an expressiveness problem nobody has characterised.

P5: Persistent compromise has no detection story. Memory and knowledge-base poisoning [62,63] and training-time backdoors [52,53,94] produce systems that behave normally until a trigger fires. There is no anomaly in the traffic to detect, and Table 4 shows the coverage: essentially none. Provenance tracking over the memory store is the obvious direction and is largely unexplored [120].

P6: Alignment is not a security boundary, and is sometimes treated as one. Refusal training does not transfer to the agentic setting [76], and models can strategically feign alignment [109] or deceive under pressure [110]. A safety-trained model is a useful defense-in-depth layer and a catastrophic sole control. Deployments that rely on the model's disposition rather than on enforced authority limits are relying on the wrong thing.

P7: Availability and cost attacks are unstudied. Asset A3 in our taxonomy is nearly empty. An agent that can be induced to loop, to invoke expensive tools repeatedly, or to exhaust a rate limit is an attractive denial-of-wallet target, and we found essentially no systematic work on it.

P8: Benchmarks are shallow relative to deployments. Real agents run in environments with credentials, side effects, multiple users, and irreversible actions. Benchmarks run in sandboxes with none of these. The construct-validity gap – does a high AgentDojo score predict a safe production deployment? – is unmeasured, and the honest answer at present is that nobody knows [121,122].

11. A research agenda

We turn the open problems into directions, ordered by what we judge to be their expected value.

11.1. Extend information-flow control across agent boundaries

The CaMeL result [24] shows that a single agent's tool calls can be gated by capabilities derived from data provenance, yielding security properties that hold regardless of whether the model was injected. The obvious and unbuilt generalisation is to make provenance labels *travel with inter-agent messages*: if agent A reads a web page, every value A derives from it carries a taint, and when A sends a message to B, the taint crosses with it, so B's tool calls are gated by a policy that knows the content originated outside the trust boundary. This directly attacks P3, it composes with the existing single-agent machinery [26,27], and the classical theory is available off the shelf [56]. The hard parts are precision (labels accumulate and over-taint) and declassification (some untrusted content legitimately must influence action—the policy needs a principled release mechanism). We regard this as the most important open direction in the field.

11.2. Topology and architecture as security controls

NetSafe [67] shows that the communication graph determines propagation dynamics. This suggests treating multi-agent architecture as a security design problem with quantifiable properties—graph structures that provably bound the infection reach of a single compromised node, delegation hierarchies where authority strictly decreases with depth, and mediated message buses in the spirit of IsolateGPT's inter-app boundaries [25]. This line would also let the security tax [51] be measured rather than merely asserted: for a given topology, what is the capability cost of a given containment guarantee? [123,124]

The formal apparatus for this exists and is not being used. The distributed-control literature has spent two decades characterising how information and influence propagate over communication graphs under exactly the conditions that agentic deployments exhibit—switching topologies, asynchronous updates, communication delays, and signed edges representing antagonistic as well as cooperative relations. Convergence-rate analyses based on products of substochastic matrices give delay-dependent bounds on how fast influence traverses a network [47], and tracking results over asynchronous cooperation–competition networks show that guarantees survive the presence of structurally antagonistic nodes [48]. The transfer is not immediate, for the reason set out in Section 2: in those settings the receiving agent's update rule is fixed and a hostile neighbour can only perturb the value it consumes, whereas an LLM agent's update rule is rewritable by the message itself, so the contraction arguments underpinning consensus bounds do not apply unmodified. But the *questions* transfer exactly, and they are the questions multi-agent agentic security has not yet learned to ask: for a given graph, what is the reach of a single compromised node as a function of hop count; which topologies are k-containing in the sense that no single compromise reaches more than k agents; and how does asynchrony or delay change the answer? A productive formulation is to model infection as a spreading process on the communication graph with a per-edge transmission probability determined by the recipient's defenses, and then to ask which graphs bound expected reach—a question for which the substochastic machinery is directly suited once the transmission model is fixed. We regard the absence of this connection as a missed opportunity on both sides.

A related body of work, further from ours but pointed to by the same reviewer, is multi-vehicle motion planning in robotics, where a team of physical agents must coordinate under topology, resource, and safety constraints in the presence of obstacles and, in the adversarial variants, other agents actively working against the team. Recent results give real-time collision-free planning for multiple underwater vehicles in dense obstacle fields [125], group-based policy optimisation for multi-group encirclement and obstacle avoidance among surface vehicles [126], and energy-constrained collaborative planning across heterogeneous amphibious vehicle teams [127]. These are motion-planning results, not security results, and we do not present them as transferable defenses; the threat model is absent (there is no analogue of an adversary rewriting a vehicle's objective through its sensor feed, which is precisely our Boundary 1 problem). What does carry over is the framing, and it strengthens the case made above: each of these results treats *heterogeneity* and a *resource budget* (battery, energy, communication range) as first-class constraints on a coordination policy over a graph, rather than assuming a homogeneous team with unlimited coordination bandwidth. That is exactly the situation a real multi-agent LLM deployment is in and that no benchmark in Section 11.4 yet captures—agents in practice are heterogeneous in model, tooling, and hardening level, and defending all of them uniformly is rarely the deployable option. We take this as independent, if indirect, motivation for requirement (iii) of the benchmark specification above: pluggable *per-agent* defenses under a resource or deployment-cost budget, rather than a single defense applied uniformly across the topology.

11.3. Provenance for memory and retrieval

Against persistent compromise (P5), the tractable direction is to make the memory store provenance-aware: every stored item records where it came from, which agent wrote it, and under what trust level, so that retrieval can be filtered by policy and a poisoned item cannot silently acquire the authority of a trusted one. This attacks AgentPoison [63] and PoisonedRAG [62] at the mechanism rather than the symptom, and it is a natural extension of the same information-flow substrate as Section 11.1.

11.4. Adaptive-by-construction evaluation

The field should adopt the AgentDojo design [17] as the default: environments where attack and defense are pluggable and the attacker is permitted to adapt. Static test sets should be understood as lower bounds on attack success, never as evidence of robustness. Benchmarks for the multi-agent case – where propagation, not single-step hijack, is the quantity of interest – do not exist and are needed; the natural metric is not ASR but *infection reach*: the fraction of the agent population compromised from a single injection, as a function of topology.

To make this actionable rather than exhortatory, we specify what such a benchmark would have to contain. A multi-agent security benchmark should expose: (i) a *configurable topology*—at minimum chain, star, mesh, and hierarchical delegation, since these differ in propagation dynamics [67]; (ii) a *benign task suite* defined over the same topology, so that infection reach and collaborative task success are measured on one system and the security tax [51] becomes a curve rather than an assertion; (iii) *pluggable per-agent defenses*, so that heterogeneous deployments – where only some agents are hardened – can be evaluated, which is the realistic case and is currently unstudied; (iv) an *adaptive attacker interface* with a declared query budget [128]; and (v) *deterministic replay*, since propagation experiments over stochastic agents are otherwise irreproducible.

We propose four primary metrics, all of which are functions of the topology G and the defense assignment D, and none of which is meaningful alone:

- **Infection reach** $R(G, D)$: expected fraction of agents compromised from a single injection at a uniformly random node, reported with its distribution and not only its mean, since the tail is the deployment risk.
- **Time-to-containment** $T(G, D)$: message-passing rounds until the infected set stops growing, or ∞ if it does not [129].

- **Collaborative utility** $U(G, D)$: benign task success on the same configuration. The pair (R, U) is the result; neither number alone is.
- **Containment cost** $\partial U/\partial R$: the local slope of the security/utility frontier, which is the quantity a practitioner actually needs and which no current work reports.

A benchmark meeting this specification would also settle a question the present survey can only pose: whether topological damping is a genuine control or merely delays the inevitable. Under the metrics above the distinction is sharp – damping reduces R at fixed rounds, whereas true containment makes T finite and R bounded independently of the number of rounds – and it is currently undecided because T is never measured.

11.5. Reproducibility of agent-security results

Agentic security experiments are unusually difficult to reproduce, and the difficulty is not incidental. Results depend on a specific model checkpoint behind an API that is silently updated, on a stochastic decoding configuration, on an environment that may reach the live web, and frequently on an LLM judge that is itself a moving target. In assembling Table 3 we found that even where artefacts were released (29 of 44 attack papers), re-deriving the reported headline number was rarely possible from the artefact alone.

Four requirements would remove most of this, and none is burdensome. *Pin the model*: report the exact checkpoint identifier and the date of access, since a version string alone does not identify behaviour for a continuously served model. *Pin the decoding*: report temperature, top-p, seed where the provider supports it, and the number of independent runs, with dispersion—single-run ASR figures on stochastic agents are not measurements. *Pin the environment*: prefer a hermetic, snapshotted environment over the live web, and where live access is essential, record the retrieved content so the trajectory can be replayed offline [130]. *Pin the judge*: release the judge prompt and its checkpoint, and report agreement with human labels on a sample, since an unvalidated judge that is itself inside the attacker's reach [98] is an instrument of unknown calibration measuring an adversarially chosen quantity. We fold these into the reporting standard below.

11.6. Availability and denial-of-wallet

A neglected but wide-open direction (P7): systematise the attacks that make an agent expensive rather than dangerous, and the budget-enforcement mechanisms that bound them [131].

11.7. A minimal security-reporting standard

Much of the incommensurability documented in Section 9 is fixable by convention. We propose that papers presenting an attack or a defense on an agentic system report, as a structured block:

1. **Attacker capability.** What the attacker controls, chosen explicitly from: model weights, training data, scaffold, tool implementations, tool metadata, environment content, a peer agent, the user channel. State what is *not* controlled.
2. **Defender scope.** What the defense may modify: prompt only, scaffold, model weights, system architecture. A prompt-only defense and an architecture-level one are not competitors.
3. **Paired (ASR, utility).** Attack success rate *and* benign task success under the same defense, on the same environment. Neither number alone is a result.
4. **Adaptivity.** Whether the attacker was permitted to adapt to the defense. If not, report the result as an upper bound on defense strength.

Table 6
The reporting standard of Section 11.7 applied to four representative studies. Columns 1–9 are the numbered items above. • satisfied, ∘ partial (present but not fully specified), — absent. AgentDojo and CaMeL are the strongest reporters in the corpus by appraisal score (Section 3.4); InjecAgent is a typical mid-corpus attack paper; Greshake et al. is foundational but, being an early demonstration rather than a benchmark, is sparse by the standards proposed here—which the table is intended to show honestly rather than to paper over.

Study	1	2	3	4	5	6	7	8	9
AgentDojo [17]	•	•	•	•	∘	–	•	∘	∘
CaMeL [24]	•	•	•	•	–	–	•	∘	•
InjecAgent [16]	•	∘	–	–	∘	–	•	–	–
Greshake et al. [9]	∘	–	–	–	–	–	∘	–	–

5. **Judge and its threat status.** How success was scored, and – if an LLM judge – whether the judge was inside the attacker's reach [98].
6. **Cost.** Attacker and defender compute/token budgets [116].
7. **Taxonomic coordinates.** Entry point, propagation path, target asset (Section 4), so that the contribution can be located against the coverage map.
8. **Reproducibility block.** Model checkpoint and access date; decoding parameters, seed, and number of runs with dispersion; whether the environment is hermetic or live, and if live, whether trajectories are recorded for replay (Section 11.5).
9. **Residual risk.** For a defense: what remains exploitable when the defense operates as intended, and which assumption, if violated, voids the guarantee. A defense paper that cannot answer this has not stated a threat model. [132]

None of this is expensive. All of it is currently missing often enough that the literature cannot be aggregated, which is the difference between a field that accumulates knowledge and one that accumulates papers. To lower the cost of adoption further, items 1–3 and 7 can be reported as a single structured table of at most fifteen rows.

A standard proposed but never applied to the survey's own material is easy to dismiss as aspirational, so we apply it here to four representative studies spanning the corpus, chosen to include a strong reporter, a typical one, and a sparse one rather than only best cases. Table 6 scores each against the nine items above, using • for satisfied, ∘ for partial, and — for absent, read directly from the entries already recorded in Table 3 and Section 3.4 rather than re-derived.

The pattern is the point rather than any single cell. Item 6 (cost) is absent for all four, confirming that this is not a gap peculiar to weaker papers but a field-wide omission (Section 9.2); items 8–9 (reproducibility block, residual risk), being new proposals of this survey, are naturally unmet by prior work and are shown here only so that future submissions have a visible bar to clear. Applying the standard to our own comparative claims does not make those claims better evidenced than the sources they rest on – it cannot – but it does make the shortfall specific rather than general, which is the reporting standard's actual purpose.

11.8. Maintaining this survey: a protocol for a moving field

A survey of a literature that doubles annually is obsolete on a schedule, and pretending otherwise is a disservice. Our search closed on 31 January 2026; on the observed publication rate, roughly a third of the relevant corpus at the time of reading will postdate it. Rather than treat this as an unavoidable defect we specify how the artefact should be maintained, and we commit to the procedure.

What is designed to be stable. The taxonomy (Section 4) is indexed on architectural features – where content enters, how it travels, what it violates – rather than on attack techniques, and architectures change more slowly than techniques. The constraints (D1)–(D4) are statements about system structure and should survive any particular attack going

out of fashion. The coverage criterion (Section 8.1) is likewise a rule for grading evidence, not a grading. These are the parts we expect to age well, and a new attack should be classifiable by assigning coordinates rather than by adding a category.

What requires periodic re-execution. The corpus, Tables 2–5, and the quality-appraisal distribution are snapshots. The protocol of Section 3 is stated in full precisely so that it can be re-run: the search strings are quoted verbatim, the databases and date bounds are explicit, and the inclusion criteria are operational. Re-execution with an updated upper date bound should reproduce the existing corpus as a prefix; any failure to do so is a defect in the protocol's statement and should be reported as such.

Recommended cadence and triggers. We suggest a six-month re-run of the search, with three additional event triggers that should prompt an off-cycle update regardless of the calendar: the appearance of a defense earning a ▲ grade in a currently uncovered cell of Table 4; a demonstrated break of a defense currently graded ▲, which is the update that matters most because deployments may depend on it; and the emergence of an attack whose coordinates fall outside the admissible set $\mathcal{T}$, which would falsify one of (D1)–(D4) and require revising the taxonomy rather than extending the tables. One candidate revision is already queued: splitting entry point $E5$ into a training-time sub-case and a memory/RAG sub-case (Section 4.1), which a reviewer correctly noted conflates two different attacker-capability regimes under one label. We have not made the split in this revision because it would propagate through Table 2's coordinates, constraint (D1), and every downstream citation of $E5$, which is a larger change than is safe to make without re-deriving the affected counts; it is recorded here so the next re-run of the protocol addresses it deliberately rather than by omission.

Versioning. Because a substantial fraction of the sources are preprints revised in place (Section 3.5), the corpus should be versioned with the consulted version of each source recorded, and each release of the survey should carry a changelog identifying records added, records whose grading changed, and – separately and prominently – any claim retracted. We maintain the corpus, the screening decisions with their governing criteria, and the per-cell grading rationales in a public repository so that the update can be performed by others, and so that the intra-rater screening limitation of Section 3.3 can be remedied by independent re-screening rather than merely acknowledged.

12. Conclusion

The security problem of LLM agents is not a new vulnerability class; it is an old one – the inability to separate instructions from data, and the confused deputy that follows – given a vastly larger blast radius by two developments: the connection of models to real tools, and the composition of models into systems that trust each other's output. An injection that against a chatbot produces a bad sentence, against a tool-using agent produces an unauthorised action with the user's authority, and against a multi-agent system produces a self-propagating compromise. This survey organised that landscape along three dimensions – where adversarial content enters, how far it travels, and what it violates – and used them to review 102 papers spanning the attacks (indirect injection, malicious tool use, inter-agent propagation, memory poisoning), the defenses (detection, structural separation, isolation, information-flow control, alignment-side hardening), and the evaluations that connect them.

Two conclusions carry the most weight, and we state them with the qualification their evidence base warrants: as Section 3.5 makes explicit, roughly half of the corpus behind them is preprint material, so both are read most safely as the best current characterisation of the literature rather than as settled fact.

The first is that the durable defenses are the ones that stop trying to make the model trustworthy and instead build systems in which the model's untrustworthiness is contained—least privilege, execution isolation, and information-flow control, the classical answers, rediscovered. Every prompt-level defense we reviewed that has been tested against an adaptive attacker was defeated or substantially degraded (Section 9.4); none in the corpus is reported to withstand one. This is a claim about the published record at the close of our search window, not a proof that no such defense can exist. Every defense with a genuine guarantee assumes the model is compromised and bounds the damage anyway. The second is that this containment story exists only for the single-agent case. Multi-agent propagation – the mechanism that makes agentic compromise qualitatively worse than model compromise – remains, as the coverage map shows, essentially undefended *in the literature we found*, and extending information-flow control across agent boundaries is the field's most valuable open problem.

Enterprise adoption of agents is blocked on the ability to state a defensible security posture, and it should be, because for the multi-agent deployments that are the industry's ambition, we found no published defense yet closing that gap. Closing it is not primarily a modelling problem. It is a systems-security problem – provenance, capabilities, mediated boundaries, and honest measurement – and the tools to solve it are, for the most part, already on the shelf. What remains is to build them for agents, and to hold the field to the discipline of measuring whether they work against an attacker who is trying.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Claude Opus 5 and Claude Fable 5.1 (Anthropic) in order to improve the language and readability of the manuscript and to assist with LaTeX formatting. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.
[2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.
[3] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, et al., A survey on large language model based autonomous agents, Front. Comput. Sci. 18 (6) (2024) 186345.
[4] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, et al., The rise and potential of large language model based agents: A survey, 2023, arXiv preprint arXiv:2309.07864.
[5] Anthropic, Introducing the model context protocol, 2024, https://www.anthropic.com/news/model-context-protocol.
[6] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, et al., AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023, arXiv preprint arXiv:2308.08155.
[7] S. Hong, M. Zhuge, J. Chen, X. Zheng, et al., MetaGPT: Meta programming for a multi-agent collaborative framework, in: International Conference on Learning Representations(ICLR), 2024.
[8] C. Qian, W. Liu, H. Liu, N. Chen, et al., ChatDev: Communicative agents for software development, in: Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024.
[9] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79–90.

[10] S. Cohen, R. Bitton, B. Nassi, Here comes the AI worm: Unleashing zero-click worms that target GenAI-powered applications, 2024, arXiv preprint arXiv:2403.02817.

[11] D. Lee, M. Tiwari, Prompt infection: LLM-to-LLM prompt injection within multi-agent systems, 2024, arXiv preprint arXiv:2410.07283.

[12] X. Gu, X. Zheng, T. Pang, C. Du, et al., Agent smith: A single image can jailbreak one million multimodal LLM agents exponentially fast, in: Proc. 41st International Conference on Machine Learning (ICML), 2024.

[13] F. Perez, I. Ribeiro, Ignore previous prompt: Attack techniques for language models, in: NeurIPS ML Safety Workshop, 2022.

[14] Y. Ruan, H. Dong, A. Wang, S. Pitis, et al., Identifying the risks of LM agents with an LM-emulated sandbox, in: International Conference on Learning Representations (ICLR), 2024.

[15] S. Naihin, D. Atkinson, M. Green, M. Hamadi, et al., Testing language model agents safely in the wild, 2023, arXiv preprint arXiv:2311.10538.

[16] Q. Zhan, Z. Liang, Z. Ying, D. Kang, InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents, in: Findings of the Association for Computational Linguistics (ACL Findings), 2024.

[17] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, F. Tramèr, AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, in: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks, 2024.

[18] M. Andriushchenko, A. Souly, M. Dziemian, D. Duenas, et al., AgentHarm: A benchmark for measuring harmfulness of LLM agents, in: International Conference on Learning Representations (ICLR), 2025.

[19] H. Zhang, J. Huang, K. Mei, Y. Yao, et al., Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents, in: International Conference on Learning Representations (ICLR), 2025.

[20] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, C.N.-R. Ilie, et al., WASP: Benchmarking web agent security against prompt injection attacks, 2025, arXiv preprint arXiv:2504.18575.

[21] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, E. Kiciman, Defending against indirect prompt injection attacks with spotlighting, 2024, arXiv preprint arXiv:2403.14720.

[22] G. Alon, M. Kamfonas, Detecting language model attacks with perplexity, 2023, arXiv preprint arXiv:2308.14132.

[23] Y. Liu, Y. Jia, R. Geng, J. Jia, N.Z. Gong, Formalizing and benchmarking prompt injection attacks and defenses, in: Proc. 33rd USENIX Security Symposium, 2024.

[24] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, et al., Defeating prompt injections by design, 2025, arXiv preprint arXiv:2503.18813.

[25] Y. Wu, F. Roesner, T. Kohno, N. Zhang, U. Iqbal, IsolateGPT: An execution isolation architecture for LLM-based agentic systems, in: Proc. Network and Distributed System Security Symposium (NDSS), 2025.

[26] F. Wu, E. Cecchetti, C. Xiao, System-level defense against indirect prompt injection attacks: An information flow control perspective, 2024, arXiv preprint arXiv:2409.19091.

[27] M. Costa, S.Z. Béguelin, C. Fournet, P. Kohli, et al., Securing AI agents with information-flow control, 2025, arXiv preprint arXiv:2505.23643.

[28] J.H. Saltzer, M.D. Schroeder, The protection of information in computer systems, Proc. IEEE 63 (9) (1975) 1278–1308.

[29] N. Hardy, The confused deputy: (or why capabilities might have been invented), ACM SIGOPS Oper. Syst. Rev. 22 (4) (1988) 36–38.

[30] OWASP Foundation, OWASP top 10 for large language model applications (2025), 2025, https://owasp.org/www-project-top-10-for-large-language-model-applications/.

[31] OWASP Foundation, OWASP agentic AI threats and mitigations, 2025, https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/.

[32] National Institute of Standards and Technology, Artificial intelligence risk management framework: Generative artificial intelligence profile (NIST AI 600-1), 2024, NIST.

[33] MITRE, MITRE ATLAS: Adversarial threat landscape for artificial-intelligence systems, 2024, https://atlas.mitre.org/.

[34] Y. Liu, Y. Yao, J.-F. Ton, X. Zhang, et al., Trustworthy LLMs: A survey and guideline for evaluating large language models' alignment, 2023, arXiv preprint arXiv:2308.05374.

[35] Y. Chang, X. Wang, J. Wang, Y. Wu, et al., A survey on evaluation of large language models, ACM Trans. Intell. Syst. Technol. 15 (3) (2024) 1–45.

[36] Z. Deng, Y. Guo, C. Han, W. Ma, et al., AI agents under threat: A survey of key security challenges and future pathways, ACM Comput. Surv. (2025).

[37] S. Rossi, A.M. Michel, R.R. Mukkamala, J.B. Thatcher, An early categorization of prompt injection attacks on large language models, 2024, arXiv preprint arXiv:2402.00898.

[38] F. He, T. Zhu, D. Ye, B. Liu, W. Zhou, P.S. Yu, The emerged security and privacy of LLM agent: A survey with case studies, 2024, arXiv preprint arXiv:2407.19354.

[39] Y. Li, H. Wen, W. Wang, X. Li, et al., Personal LLM agents: Insights and survey about the capability, efficiency and security, 2024, arXiv preprint arXiv:2401.05459.

[40] Y. Gan, Z. Yang, Z. Yue, W. Sun, Y.S. Chen, Navigating the risks: A survey of security, privacy, and ethics threats in LLM-based agents, 2024, arXiv preprint arXiv:2411.09523.

[41] M. Yu, F. Meng, X. Zhou, S. Wang, et al., A survey on trustworthy LLM agents: Threats and countermeasures, 2025, arXiv preprint arXiv:2503.09648.

[42] K. Wang, G. Zhang, Z. Zhou, J. Wu, M. Yu, et al., A comprehensive survey in LLM(-agent) full stack safety: Data, training and deployment, 2025, arXiv preprint arXiv:2504.15585.

[43] A. Zou, Z. Wang, N. Carlini, M. Nasr, J.Z. Kolter, M. Fredrikson, Universal and transferable adversarial attacks on aligned language models, 2023, arXiv preprint arXiv:2307.15043.

[44] A. Wei, N. Haghtalab, J. Steinhardt, Jailbroken: How does LLM safety training fail? in: Advances in Neural Information Processing Systems (NeurIPS), 2023.

[45] P. Chao, A. Robey, E. Dobriban, H. Hassani, G.J. Pappas, E. Wong, Jailbreaking black box large language models in twenty queries, 2023, arXiv preprint arXiv:2310.08419.

[46] T. Guo, X. Chen, Y. Wang, R. Chang, et al., Large language model based multi-agents: A survey of progress and challenges, in: Proc. 33rd Int. Joint Conf. on Artificial Intelligence (IJCAI), 2024.

[47] L. Shi, S. Yan, W. Li, Consensus and products of substochastic matrices: Convergence rate with communication delays, IEEE Trans. Syst. Man, Cybern.: Syst. 55 (7) (2025) 4752–4761, http://dx.doi.org/10.1109/TSMC.2025.3559667.

[48] W. Li, S. Yan, L. Shi, J. Yue, M. Shi, B. Lin, K. Qin, Multiagent consensus tracking control over asynchronous cooperation–competition networks, IEEE Trans. Cybern. 55 (9) (2025) 4347–4360, http://dx.doi.org/10.1109/TCYB.2025.3583387.

[49] S. Willison, The lethal trifecta for AI agents: Private data, untrusted content, and external communication, 2025, Blog post, https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/.

[50] T. Ju, Y. Wang, X. Ma, P. Cheng, et al., Flooding spread of manipulated knowledge in LLM-based multi-agent communities, 2024, arXiv preprint arXiv:2407.07791.

[51] P. Peigné, M. Kniejski, F. Sondej, M. David, et al., Multi-agent security tax: Trading off security and collaboration capabilities in multi-agent systems, in: Proc. AAAI Conference on Artificial Intelligence, 2025.

[52] Y. Wang, D. Xue, S. Zhang, S. Qian, BadAgent: Inserting and activating backdoor attacks in LLM agents, in: Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024.

[53] W. Yang, X. Bi, Y. Lin, S. Chen, J. Zhou, X. Sun, Watch out for your agents! investigating backdoor threats to LLM-based agents, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.

[54] N. Carlini, M. Jagielski, C.A. Choquette-Choo, D. Paleka, et al., Poisoning web-scale training datasets is practical, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2024.

[55] J.-t. Huang, J. Zhou, T. Jin, X. Zhou, et al., On the resilience of multi-agent systems with malicious agents, 2024, arXiv preprint arXiv:2408.00989.

[56] D.E. Denning, A lattice model of secure information flow, Commun. ACM 19 (5) (1976) 236–243.

[57] X. Hou, Y. Zhao, S. Wang, H. Wang, Model context protocol (MCP): Landscape, security threats, and future research directions, 2025, arXiv preprint arXiv:2503.23278.

[58] B. Radosevich, J. Halloran, MCP safety audit: LLMs with the model context protocol allow major security exploits, 2025, arXiv preprint arXiv:2504.03767.

[59] F. Wu, S. Wu, Y. Cao, C. Xiao, WIPI: A new web threat for LLM-driven web agents, 2024, arXiv preprint arXiv:2402.16965.

[60] Z. Liao, L. Mo, C. Xu, M. Kang, et al., EIA: Environmental injection attack on generalist web agents for privacy leakage, in: International Conference on Learning Representations (ICLR), 2025.

[61] A. Amayuelas, X. Yang, A. Antoniades, W. Hua, et al., MultiAgent collaboration attack: Investigating adversarial attacks in large language model collaborations via debate, in: Findings of the Association for Computational Linguistics (EMNLP Findings), 2024.

[62] W. Zou, R. Geng, B. Wang, J. Jia, PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models, in: Proc. 34th USENIX Security Symposium, 2025.

[63] Z. Chen, Z. Xiang, C. Xiao, D. Song, B. Li, AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.

[64] E. Bagdasaryan, R. Yi, S. Ghalebikesabi, P. Kairouz, et al., AirGapAgent: Protecting privacy-conscious conversational agents, in: Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS), 2024.

[65] S. Abdelnabi, K. Greshake, S. Mishra, C. Endres, T. Holz, M. Fritz, Are you still on track!? Catching LLM task drift with activations, 2024, arXiv preprint arXiv:2406.00799.

[66] S. Abdelnabi, A. Fay, G. Cherubin, A. Salem, M. Fritz, A. Paverd, Get my drift? Catching LLM task drift with Activation Deltas, in: Proc. IEEE Conf. on Secure and Trustworthy Machine Learning (SaTML), 2025.

[67] M. Yu, S. Wang, G. Zhang, J. Mao, et al., NetSafe: Exploring the topological safety of multi-agent networks, 2024, arXiv preprint arXiv:2410.15686.

[68] S. Schulhoff, J. Pinto, A. Khan, L.-F. Bouchard, et al., Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global scale prompt hacking competition, in: Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP), 2023.

[69] S. Toyer, O. Watkins, E. Mendes, J. Svegliato, et al., Tensor Trust: Interpretable prompt injection attacks from an online game, in: International Conference on Learning Representations (ICLR), 2024.

[70] C. Xu, M. Kang, J. Zhang, Z. Liao, et al., AdvWeb: Controllable black-box attacks on VLM-powered web agents, 2024, arXiv preprint arXiv:2410.17401.

[71] H. Kim, M. Song, S.H. Na, S. Shin, K. Lee, When LLMs go online: The emerging threat of web-enabled LLMs, 2024, arXiv preprint arXiv:2410.14569.

[72] I. Nakash, G. Kour, G. Uziel, A. Anaby-Tavor, Breaking ReAct agents: Foot-in-the-door attack will get you in, 2024, arXiv preprint arXiv:2410.16950.

[73] M. Russinovich, A. Salem, R. Eldan, Great, now write an article about that: The crescendo multi-turn LLM jailbreak attack, in: Proc. 34th USENIX Security Symposium, 2025.

[74] A. Mehrotra, M. Zampetakis, P. Kassianik, B. Nelson, H. Anderson, Y. Singer, A. Karbasi, Tree of attacks: Jailbreaking black-box LLMs automatically, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.

[75] M. Andriushchenko, F. Croce, N. Flammarion, Jailbreaking leading safety-aligned LLMs with simple adaptive attacks, in: International Conference on Learning Representations (ICLR), 2025.

[76] P. Kumar, E. Lau, S. Vijayakumar, T. Trinh, et al., Refusal-trained LLMs are easily jailbroken as browser agents, 2024, arXiv preprint arXiv:2410.13886.

[77] A.D. Tur, X.H. Lù, J. Rodriguez, F.F. Xu, et al., SafeArena: Evaluating the safety of autonomous web agents, 2025, arXiv preprint arXiv:2503.04957.

[78] R. Fang, R. Bindu, A. Gupta, Q. Zhan, D. Kang, LLM agents can autonomously hack websites, 2024, arXiv preprint arXiv:2402.06664.

[79] R. Fang, R. Bindu, A. Gupta, D. Kang, LLM agents can autonomously exploit one-day vulnerabilities, 2024, arXiv preprint arXiv:2404.08144.

[80] T. Yuan, Z. He, L. Dong, Y. Wang, et al., R-Judge: Benchmarking safety risk awareness for LLM agents, in: Findings of the Association for Computational Linguistics (EMNLP Findings), 2024.

[81] S. Zhou, F.F. Xu, H. Zhu, X. Zhou, et al., WebArena: A realistic web environment for building autonomous agents, in: International Conference on Learning Representations (ICLR), 2024.

[82] V.S. Narajala, I. Habler, Enterprise-grade security for the model context protocol (MCP): Frameworks and mitigation strategies, 2025, arXiv preprint arXiv:2504.08623.

[83] F. Baiardi, V. Sammartino, Quantifying the impact of CVSS score ordering on attack paths, in: Proc. 12th EAI Int. Conf. on Smart Objects and Technologies for Social Good, GOODTECHS, 2026.

[84] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, et al., PentestGPT: Evaluating and harnessing large language models for automated penetration testing, in: Proc. 33rd USENIX Security Symposium, 2024.

[85] A.K. Zhang, N. Perry, R. Dulepet, J. Ji, et al., Cybench: A framework for evaluating cybersecurity capabilities and risks of language models, in: International Conference on Learning Representations (ICLR), 2025.

[86] M. Bhatt, S. Chennabasappa, Y. Li, C. Nikolaidis, et al., CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models, 2024, arXiv preprint arXiv:2404.13161.

[87] M. Phuong, M. Aitchison, E. Catt, S. Cogan, et al., Evaluating frontier models for dangerous capabilities, 2024, arXiv preprint arXiv:2403.13793.

[88] M. Kinniment, L.J.K. Sato, H. Du, B. Goodrich, et al., Evaluating language-model agents on realistic autonomous tasks, 2023, arXiv preprint arXiv:2312.11671.

[89] Y. Tian, X. Yang, J. Zhang, Y. Dong, H. Su, Evil geniuses: Delving into the safety of LLM-based agents, 2023, arXiv preprint arXiv:2311.11855.

[90] Z. Zhang, Y. Zhang, L. Li, H. Gao, et al., PsySafe: A comprehensive framework for psychological-based attack, defense, and evaluation of multi-agent system safety, Proc. 62nd Annu. Meet. Assoc. Comput. Linguist. (ACL) (2024).

[91] H. Nissenbaum, Privacy as contextual integrity, Wash. Law Rev. 79 (2004) 119–157.

[92] N. Mireshghallah, H. Kim, X. Zhou, Y. Tsvetkov, et al., Can LLMs keep a secret? Testing privacy implications of language models via contextual integrity theory, Int. Conf. Learn. Represent. (ICLR) (2024).

[93] R. Staab, M. Vero, M. Balunović, M. Vechev, Beyond memorization: Violating privacy via inference with large language models, Int. Conf. Learn. Represent. (ICLR) (2024).

[94] Z. Xiang, F. Jiang, Z. Xiong, B. Ramasubramanian, R. Poovendran, B. Li, BadChain: Backdoor chain-of-thought prompting for large language models, Int. Conf. Learn. Represent. (ICLR) (2024).

[95] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, F. Wu, Benchmarking and defending against indirect prompt injection attacks on large language models, 2023, arXiv preprint arXiv:2312.14197.

[96] C.H. Wu, J.Y. Koh, R. Salakhutdinov, D. Fried, A. Raghunathan, Adversarial attacks on multimodal agents, 2024, arXiv preprint arXiv:2406.12814.

[97] R. Pedro, D. Castro, P. Carreira, N. Santos, From prompt injections to SQL injection attacks: How protected is your LLM-integrated web application?, 2023, arXiv preprint arXiv:2308.01990.

[98] J. Shi, Z. Yuan, Y. Liu, Y. Huang, et al., Optimization-based prompt injection attack to LLM-as-a-judge, Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS) (2024).

[99] H. Inan, K. Upasani, J. Chi, R. Rungta, et al., Llama Guard: LLM-based input-output safeguard for human-AI conversations, 2023, arXiv preprint arXiv:2312.06674.

[100] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, J. Cohen, NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails, Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP): Syst. Demonstr. (2023).

[101] M. Sharma, M. Tong, J. Mu, J. Wei, et al., Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming, 2025, arXiv preprint arXiv:2501.18837.

[102] S. Chen, J. Piet, C. Sitawarin, D. Wagner, StruQ: Defending against prompt injection with structured queries, in: Proc. 34th USENIX Security Symposium, 2025.

[103] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, C. Guo, SecAlign: Defending against prompt injection with preference optimization, in: Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS), 2025.

[104] J. Piet, M. Alrashed, C. Sitawarin, S. Chen, et al., Jatmo: Prompt injection defense by task-specific finetuning, in: Proc. European Symposium on Research in Computer Security (ESORICS), 2024.

[105] V. Sammartino, A framework for proactive cyber-resilience: Non-intrusive modeling for autonomous defense, in: Proc. 29th International Symposium on Distributed Simulation and Real Time Applications (DS-RT), 2025.

[106] F. Baiardi, V. Sammartino, D3F: A framework to minimize the impact of intrusions, 2024, Preprints.

[107] M. Balunović, E. Debenedetti, F. Tramèr, M. Vechev, AI agents with formal security guarantees, 2025, arXiv preprint arXiv:2506.08837.

[108] L. Beurer-Kellner, M. Fischer, M. Vechev, Prompting is programming: A query language for large language models, in: Proc. ACM on Programming Languages (PLDI), 2023.

[109] R. Greenblatt, C. Denison, B. Wright, F. Roger, et al., Alignment faking in large language models, 2024, arXiv preprint arXiv:2412.14093.

[110] J. Scheurer, M. Balesni, M. Hobbhahn, Large language models can strategically deceive their users when put under pressure, ICLR Work. Large Lang. Model. (LLM) Agents (2024).

[111] Y. Zeng, Y. Wu, X. Zhang, H. Wang, Q. Wu, AutoDefense: Multi-agent LLM defense against jailbreak attacks, 2024, arXiv preprint arXiv:2403.04783.

[112] F. Baiardi, V. Sammartino, Quantifying resilience of cyber-physical systems to zero-day threats: A security twin-based what-if analysis, in: Proc. 36th European Safety and Reliability Conference, ESREL, 2026.

[113] S. Willison, The dual LLM pattern for building AI assistants that can resist prompt injection, 2023, Blog post, https://simonwillison.net/2023/Apr/25/dual-llm-pattern/.

[114] M. Mazeika, L. Phan, X. Yin, A. Zou, et al., HarmBench: A standardized evaluation framework for automated red teaming and robust refusal, in: Proc. 41st International Conference on Machine Learning (ICML), 2024.

[115] A. Souly, Q. Lu, D. Bowen, T. Trinh, et al., A StrongREJECT for empty jailbreaks, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.

[116] S. Kapoor, B. Stroebl, Z.S. Siegel, N. Nadgir, A. Narayanan, AI agents that matter, 2024, arXiv preprint arXiv:2407.01502.

[117] B. Biggio, F. Roli, Wild patterns: Ten years after the rise of adversarial machine learning, Pattern Recognit. 84 (2018) 317–331.

[118] N. Papernot, P. McDaniel, A. Sinha, M.P. Wellman, SoK: Security and privacy in machine learning, in: Proc. IEEE European Symposium on Security and Privacy (EuroS&P), 2018.

[119] V. Sammartino, F. Baiardi, S. Ruggieri, Adversarial reliability: From design to deployment and beyond using attack-path chokepoints, 2026, Available at SSRN 7382213.

[120] F. Baiardi, S. Ruggieri, V. Sammartino, Security twins e il futuro della previsione di intrusioni cyber, ICT Secur. (2025).

[121] F. Baiardi, S. Ruggieri, V. Sammartino, Anticipating disasters through a security twin, in: Dynamics of Disasters: Hybrid Threats, 2026, pp. 1–14.

[122] V. Sammartino, F. Baiardi, Discovering when a security twin cheats: An equivalence theorem, 2026, Available at SSRN 7277218.

[123] F. Baiardi, V. Sammartino, S. Ruggieri, Graph-based cyber defence using a security twin, in: Proc. 29th International Symposium on Distributed Simulation and Real Time Applications (DS-RT), 2025.

[124] V. Sammartino, NotLine: Dynamic network topology and risk assessment through passive discovery, 2025, FAIR General Conference, Rome.

[125] X. Chen, S. Yin, Y. Li, Z. Xiang, Real-time path planning for multiple autonomous underwater vehicles in dense obstacle fields: The DA-IFDS approach, IEEE J. Ocean. Eng. 51 (3) (2026) 1787–1803, http://dx.doi.org/10.1109/JOE.2026.3674024.

[126] X. Chen, S. Yin, Y. Li, Z. Xiang, Multigroup encirclement and obstacle avoidance of unmanned surface vehicles via group-based policy optimization, IEEE Trans. Ind. Inform. (2026) http://dx.doi.org/10.1109/TII.2026.3717636, Early access.

[127] S. Yin, Z. Xiang, Energy-constrained collaborative path planning for heterogeneous amphibious unmanned surface vehicles in obstacle-cluttered environments, Ocean Eng. 330 (2025) 121241, http://dx.doi.org/10.1016/j.oceaneng.2025.121241.

[128] F. Baiardi, V. Sammartino, S. Ruggieri, Evaluating adversary strategies through a security twin, in: IEEE Int. Conf. on Pervasive Computing and Communications Workshops (PerCom Workshops), 2026.

[129] F. Baiardi, V. Sammartino, Simulation-powered cybersecurity: Real-time risk assessment via non-intrusive security twin, J. Supercomput. (2026) Special Issue on Simulation-Powered Cybersecurity.

[130] F. Baiardi, V. Sammartino, From digital twins to AI agents: A synthetic data paradigm for next-generation cybersecurity, in: Artificial Intelligence in Cybersecurity: Unlocking the Power of Large Language Models, 2026.

[131] F. Baiardi, V. Sammartino, Quantifying cyber robustness and financial risk via security twins, 2026, Available at SSRN 6890389.

[132] F. Baiardi, V. Sammartino, A quantitative framework for the validation of twin-based cyber defense, Procedia Comput. Sci. 274 (2025) 721–730.